SĂPTĂMÂNA 3

# Procese stocastice: plimbări aleatoare și lanțuri Markov

Modelare și Simulare · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

# Unde am rămas

- ✓ Distribuțiile de lucru: **Poisson** (numărăm), **exponențiala** (timpi), **normala** (medii)
- ✓ Numere **pseudo-aleatoare**, seed, **transformarea inversă**: din U(0, 1) obținem orice distribuție
- ✓ Monte Carlo: eroarea ≈ **σ/√N**; raportăm $\bar{x} \pm 1{,}96 \cdot s/\sqrt{N}$
- ✓ Azi: variabile aleatoare **legate în timp** → procese stocastice

# La finalul săptămânii veți putea…

1. să simulați o **plimbare aleatoare** și să explicați de ce distanța crește ca **√n**
2. să legați regulile **microscopice** de comportamentul **macroscopic** (difuzia)
3. să descrieți un sistem ca **lanț Markov**: stări, graf, **matrice de tranziție**
4. să calculați distribuția după **n pași** și **distribuția staționară**
5. să recunoașteți stările **absorbante**, periodice și lanțurile ergodice
6. să explicați **PageRank**, generarea de text și ideea algoritmului **Metropolis**

Ora 1

# Plimbarea aleatoare și difuzia

1. De la variabile aleatoare la procese stocastice
2. Plimbarea aleatoare 1D: legea √n
3. Plimbarea 2D și difuzia (Cernazean)
4. Unde apare: fizică, finanțe, informatică, AI generativ

# De la variabile aleatoare la procese stocastice

- Săptămâna 2: variabile aleatoare **independente** (fiecare punct Monte Carlo era „nou”)
- Acum: un **șir în timp** $X_0, X_1, X_2, \ldots$, în care valoarea de azi pornește de la cea de ieri
- Exemple: numărul de biciclete dintr-o stație, lungimea unei cozi, prețul unei acțiuni
- Ne interesează **traiectoria** și comportamentul **pe termen lung**, nu doar o valoare

**Ideea**

Un **proces stocastic** este o familie de variabile aleatoare indexate de timp. O rulare a simulării produce **o traiectorie** a procesului.

# Procesul stocastic

## Proces stocastic

O familie de variabile aleatoare **{X(t)}** indexate de timp, care descrie evoluția unui sistem supus întâmplării. Mulțimea valorilor posibile se numește **spațiul stărilor**.

*după Holban, cap. 5*

- Timpul poate fi **discret** (pași 0, 1, 2, …) sau **continuu** ($t \geq 0$)
- Stările pot fi **discrete** (0, 1, 2, … clienți) sau **continue** (o poziție, un preț)
- Modelul nostru din S1 (bike share) era deja un proces stocastic în timp discret

# Patru tipuri de procese stocastice

| | Stări discrete | Stări continue |
|---|---|---|
| **Timp discret** | **Lanț Markov, plimbare aleatoare**<br>Bike share pe minute, poziția pe o rețea, vremea zi de zi. **Tema de azi.** | **Serii de timp**<br>Temperatura zilnică, prețul la închidere, plimbarea cu pași normali. |
| **Timp continuu** | **Cozi, proces Poisson**<br>Numărul de clienți la ghișeu, cererile la un server. **Săptămâna 4.** | **Mișcarea browniană**<br>Particule în fluid, modele de prețuri în timp continuu, zgomot. |

# Scurt istoric

**1827** — **Robert Brown** vede particule din polen care se mișcă haotic în apă

**1900** — **Bachelier** modelează prețurile de la Bursa din Paris ca plimbare aleatoare

**1905** — **Einstein** explică mișcarea browniană; **Pearson** inventează termenul „random walk”

**1906–13** — **Markov** definește lanțurile; le aplică pe literele din „Evgheni Oneghin”

**1953** — **Metropolis** et al.: lanțuri Markov pentru a eșantiona distribuții (MCMC)

**1998–2020** — **PageRank** (Google); modelele de **difuzie** generează imagini

# Plimbarea aleatoare simplă (1D)

## Plimbare aleatoare simplă

Pornim din $X_0 = 0$. La fiecare pas aruncăm o monedă: $\mathbf{X_{n+1} = X_n + 1}$ cu probabilitatea p, $\mathbf{X_{n+1} = X_n - 1}$ cu probabilitatea 1 − p. Pentru p = ½ plimbarea este **simetrică**.

- Poziția după n pași este **suma** a n pași independenți: $X_n = \xi_1 + \xi_2 + \ldots + \xi_n$
- Metafora clasică: un pieton „amețit” pe o stradă lungă
- Exact ce face o simulare: aplică repetat o regulă simplă cu numere aleatoare

# Plimbarea aleatoare în NumPy

```
import numpy as np
rng = np.random.default_rng(1)

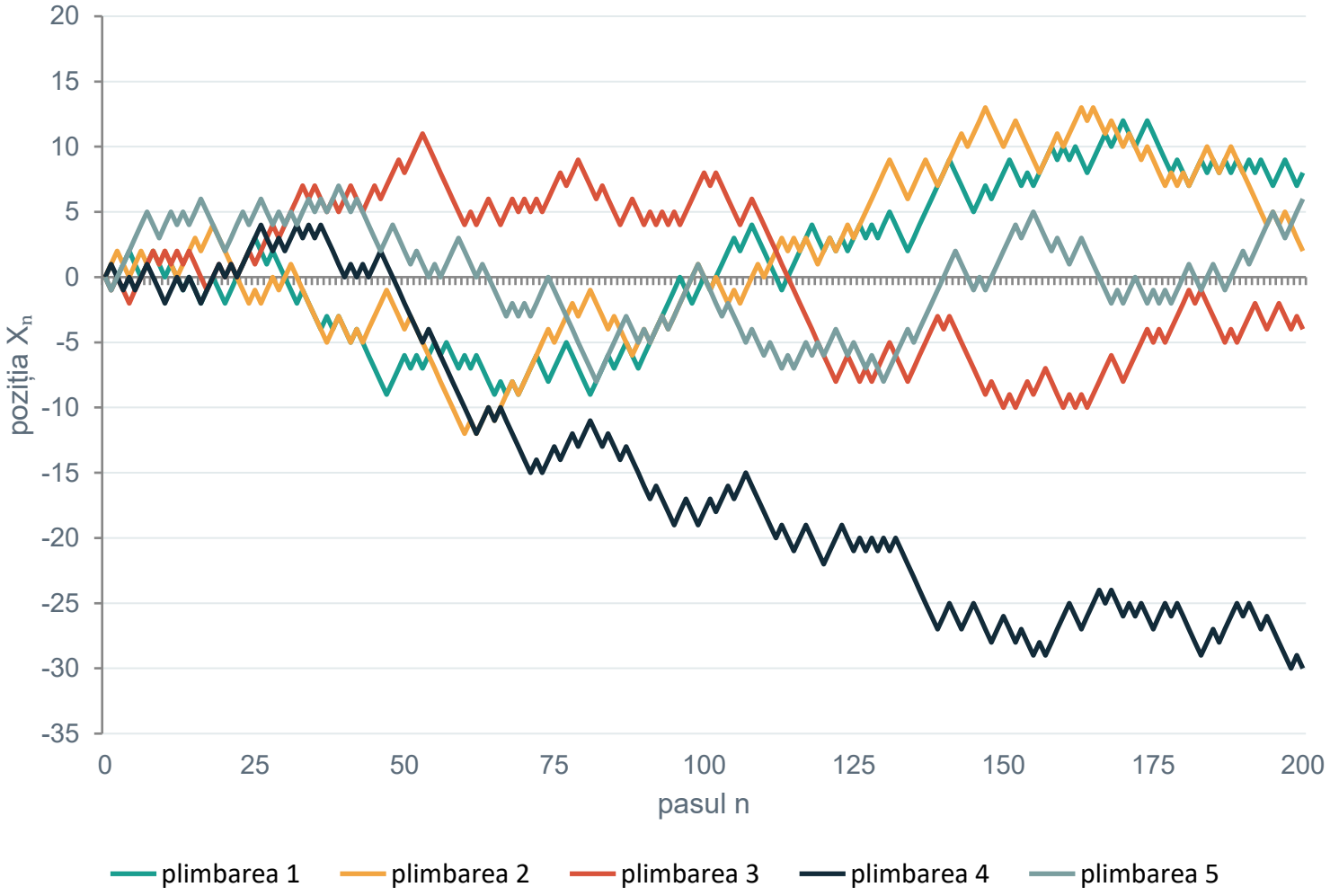
# 5 plimbări a câte 200 de pași ±1
steps = rng.choice([-1, 1], size=(5, 200))
paths = np.cumsum(steps, axis=1)   # pozițiile

print(paths[:, -1])   # unde a ajuns fiecare

# 10 000 de plimbări × 100 de pași
rng = np.random.default_rng(2)
steps = rng.choice([-1, 1], size=(10_000, 100))
x = steps.sum(axis=1)
print(x.mean(), x.std())
```

- **cumsum** = suma cumulată: poziția după fiecare pas
- O matrice: o linie = o plimbare
- Fără bucle Python: $10^6$ pași în câteva milisecunde
- Rezultat: media ≈ **0,10**, abaterea standard ≈ **9,99**

# Cinci plimbări aleatoare, 200 de pași




- Aceeași regulă, **traiectorii foarte diferite**
- Unele plimbări rămân mult timp de o parte a lui 0
- Nicio plimbare nu „știe” unde este zero
- Poziții finale între -30 și 8

# Media și dispersia poziției

$$E[X_n] = 0 \qquad Var(X_n) = n \qquad \sigma_n = \sqrt{n}$$

| | |
|---|---|
| $E[\xi] = 0$ | un pas are media 0: +1 și −1 la fel de probabile |
| $Var(\xi) = 1$ | $E[\xi^2] - 0^2 = 1$; dispersiile pașilor independenți **se adună** |
| $\sqrt{n}$ | distanța tipică față de start: după 100 de pași ≈ 10, după 10 000 ≈ 100 |

Aceeași lege √n ca eroarea Monte Carlo: suma a n variabile independente are abaterea standard σ·√n.

# Legea √n: departe, dar încet

**√100 = 10**

după **100** de pași, distanța tipică față de start

**√10 000 = 100**

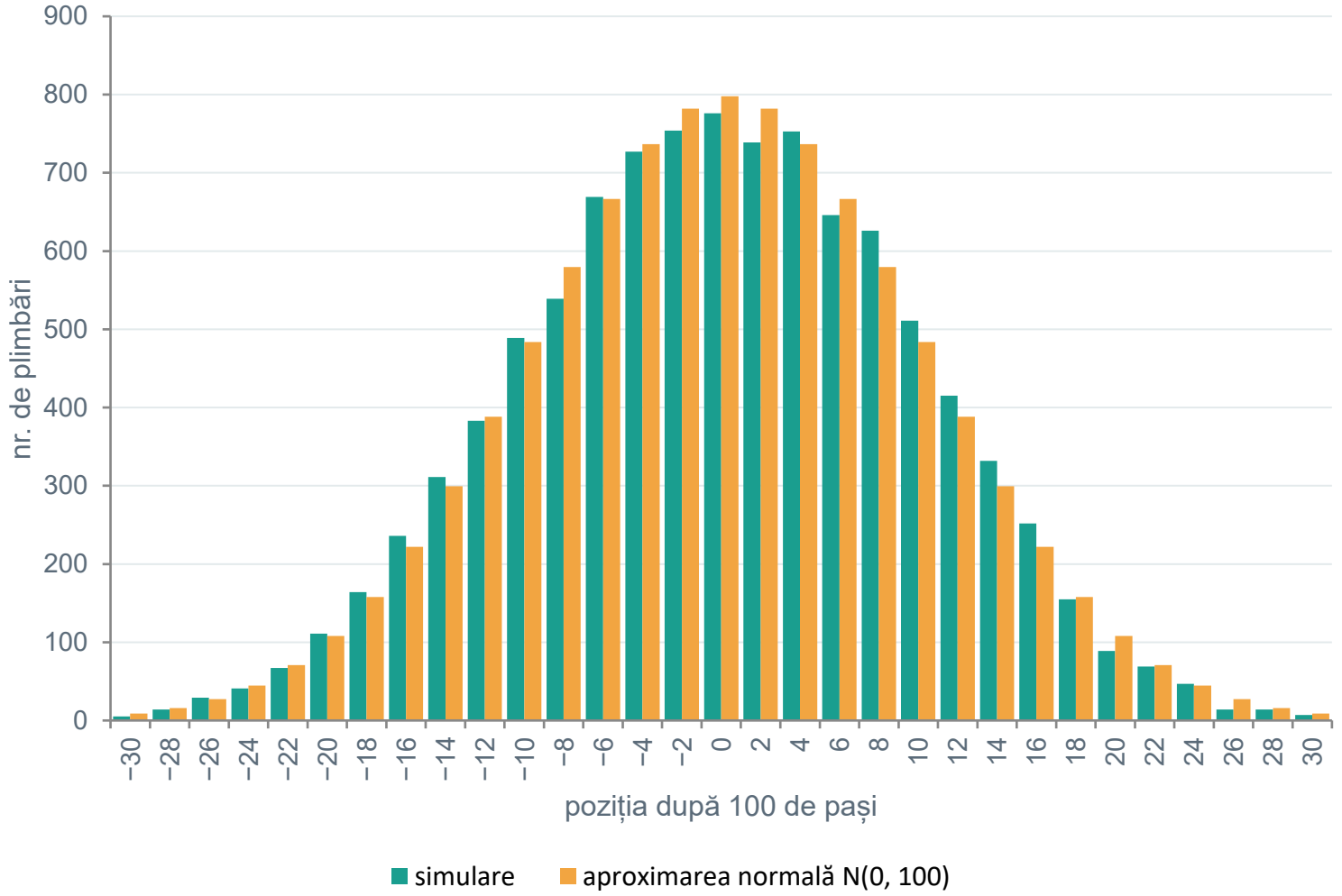
după **10 000** de pași: de 100 de ori mai mulți pași, doar de 10 ori mai departe

**8,0**

distanța medie $|X_{100}|$ măsurată în simulare (teoretic $\sqrt{(2n/\pi)} \approx 7{,}98$)

- Un pieton care merge **drept** parcurge n pași; unul „amețit” doar ≈ √n
- De aici vine încetineala difuziei (vom vedea imediat)

# Unde ajungem după 100 de pași? (10 000 de plimbări)




- Forma de **clopot**: teorema limită centrală
- Poziția este sumă de 100 de pași independenți
- Doar **16** din 10 000 de plimbări ajung mai departe de ±30 (3σ)
- Pozițiile sunt doar **pare** (100 de pași ±1)

# Verificare

? O plimbare aleatoare simetrică face **400** de pași de ±1. Care este distanța **tipică** (abaterea standard) față de punctul de plecare?

A 0

B 20

C 200

D 400

# Plimbarea cu tendință (p ≠ ½)

$$E[X_n] = n(2p - 1) \qquad Var(X_n) = 4np(1 - p)$$

| | |
|---|---|
| ***drift*** | fiecare pas are media (+1)·p + (−1)·(1 − p) = 2p − 1 |
| ***p = 0,55*** | media crește cu 0,1 pe pas: după 200 de pași, în medie +20 |
| ***zgomot*** | abaterea standard după 200 de pași: √(4·200·0,55·0,45) ≈ 14 |

Tendința crește ca **n**, zgomotul doar ca **√n**: pe termen lung, tendința câștigă. Exact principiul avantajului casei la cazino.

# Cinci plimbări cu p = 0,55




- Linia roșie: media teoretică **0,1·n**
- La început zgomotul ascunde tendința
- După ~100 de pași tendința devine vizibilă
- Lecție: o tendință mică cere **multe** date ca să fie observată

# Ruina jucătorului

$$P(\text{ajunge la } N \text{ înainte de } 0) = k / N$$

| | |
|---|---|
| $k$ | suma de start (în mize de 1 leu); jocul se oprește la 0 (ruină) sau la N (țintă) |
| $p = ½$ | joc corect: probabilitatea de câștig = k/N; durata medie = k(N − k) mize |
| $p < ½$ | $P = (1 - r^k) / (1 - r^N)$, cu $r = (1 - p)/p$ – scade foarte repede |

Primul nostru exemplu de **stări absorbante**: odată ajuns la 0 sau N, procesul se oprește (ora 2).

# Ruina jucătorului, simulată

```
import numpy as np
rng = np.random.default_rng(4)

wins, lens = 0, []
for _ in range(10_000):           # 10 000 de jocuri
    pos, n = 10, 0                # 10 lei, 0 mize
    while 0 < pos < 20:           # până la ruină/țintă
        pos += 1 if rng.random() < 0.5 else -1
        n += 1
    wins += pos == 20
    lens.append(n)

print(wins / 10_000, np.mean(lens))
```

- Aici bucla este naturală: fiecare joc are **altă durată**
- Condiția de oprire = cele două stări absorbante
- Rezultat: **0,495** și **98,9** mize
- Teoretic: 10/20 = 0,5 și 10·10 = 100

# Ruina jucătorului: ce spune simularea

**49,5%**

joc corect, 10 → 20 lei: șansa de câștig (teoretic 50%), în medie **99** mize (teoretic 100)

**36,8%**

ruletă (roșu, p = 18/37), 10 → 20 lei, mize de 1 leu

**6,3%**

ruletă, 50 → 100 lei cu mize de 1 leu (un singur pariu de 50 lei: 48,6%)

- Un dezavantaj mic (p = 0,486) devine uriaș când jocul durează mult
- Aceeași logică: sisteme cu o „pierdere” mică la fiecare pas (cozi, uzură)

# Plimbarea aleatoare în 2D (Cernazean, L3)

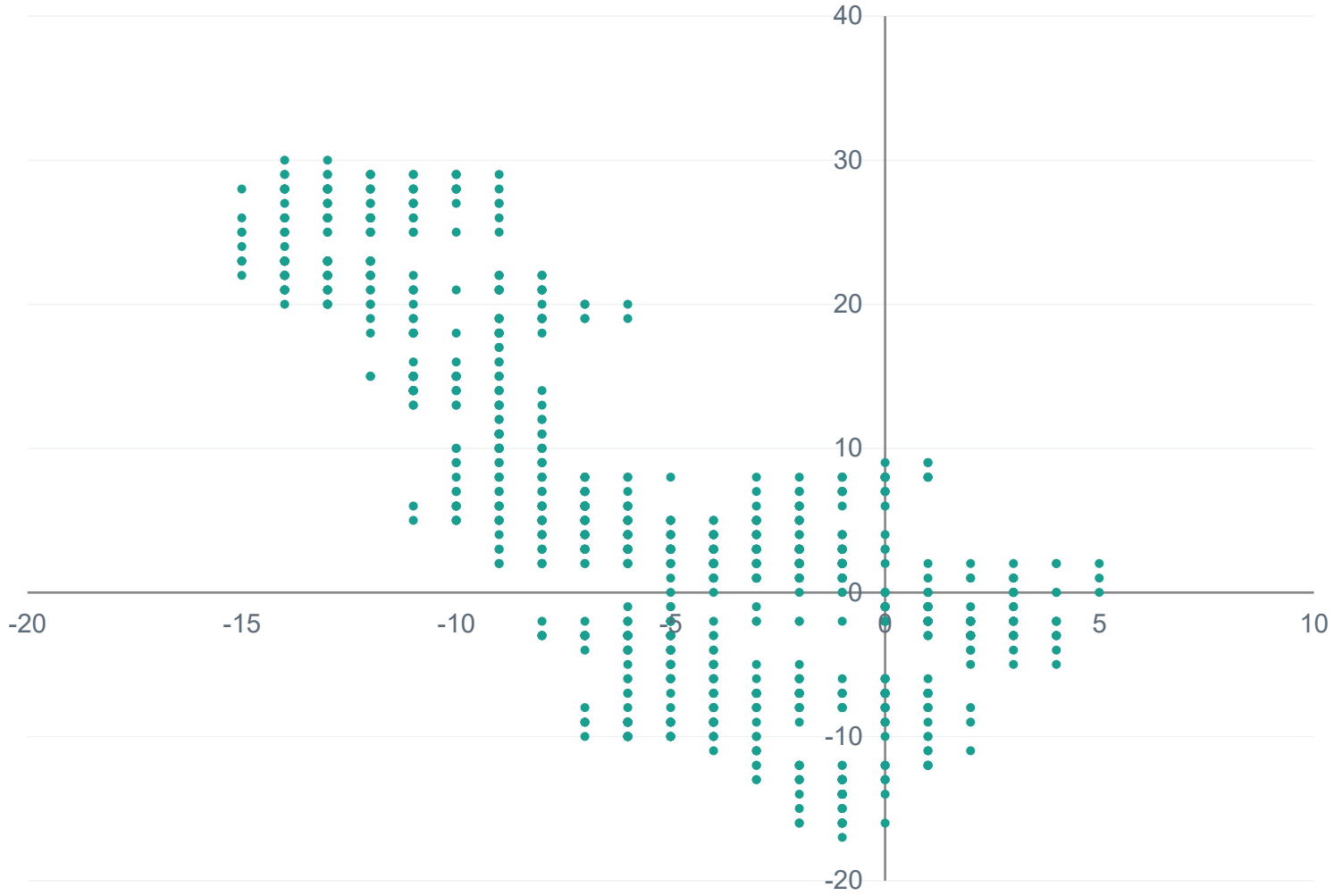
## Plimbarea pe o rețea pătrată

La fiecare pas, plimbătorul alege la întâmplare una din **5 mutări**: dreapta, stânga, sus, jos sau **pe loc**, fiecare cu probabilitatea 1/5.

*după Cernazean, Lecture 3*

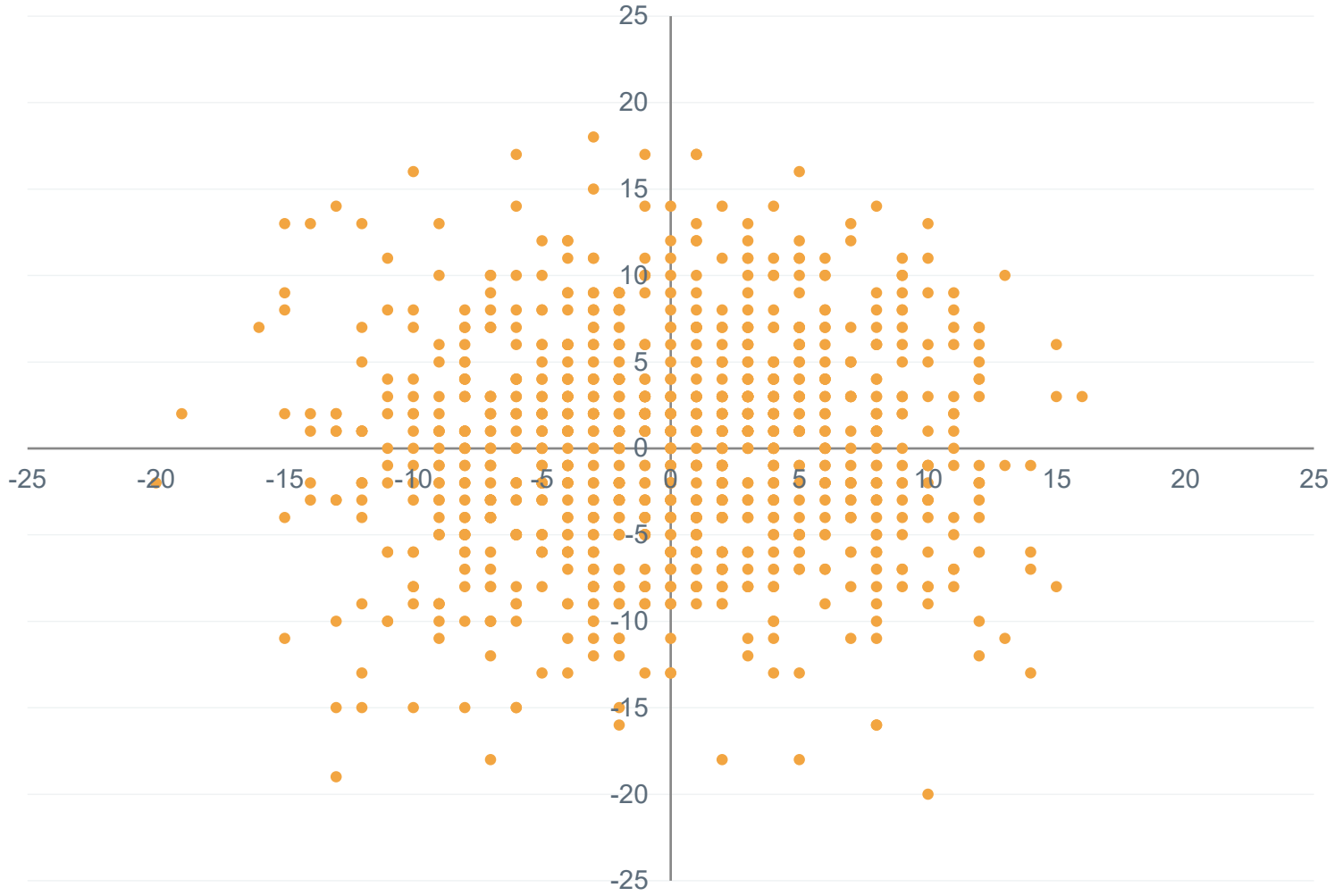
- $x(t + \Delta t) = x(t) + \Delta t \cdot v_x$, $y(t + \Delta t) = y(t) + \Delta t \cdot v_y$, cu $(v_x, v_y)$ ales aleator
- Model pentru particule, molecule, bacterii, oameni în mulțime
- Întrebarea: cum se comportă **în medie** mii de plimbători?

# Un singur plimbător, 1 000 de pași




- Traseul revine des în zone deja vizitate
- Se formează „ciorchini” și salturi mari
- Poziția finală: (-13, 26)
- Forma exactă diferă la fiecare rulare

# 1 000 de plimbători după 100 de pași




- Toți au pornit din **(0, 0)**
- Norul este **simetric**: centrul rămâne în origine
- Raza norului crește ca **√t**
- Aceasta este **difuzia**, văzută de sus

# Mii de plimbători, vectorizat

```
import numpy as np
MOVES = np.array([[1,0], [-1,0], [0,1], [0,-1], [0,0]])

rng = np.random.default_rng(7)
W, T = 10_000, 100            # plimbători, pași
k = rng.integers(0, 5, size=(W, T))
xy = np.cumsum(MOVES[k], axis=1)          # (W, T, 2)
r2 = (xy ** 2).sum(axis=2).mean(axis=0)   # ⟨r²⟩(t)

print(r2[[9, 49, 99]])   # t = 10, 50, 100
```

- **MOVES[k]**: alegem mutările prin indexare
- Un milion de pași dintr-o singură instrucțiune
- $\langle r^2 \rangle$ = media pătratului distanței față de start
- Rezultat: 8,1; 40,1; 80,1

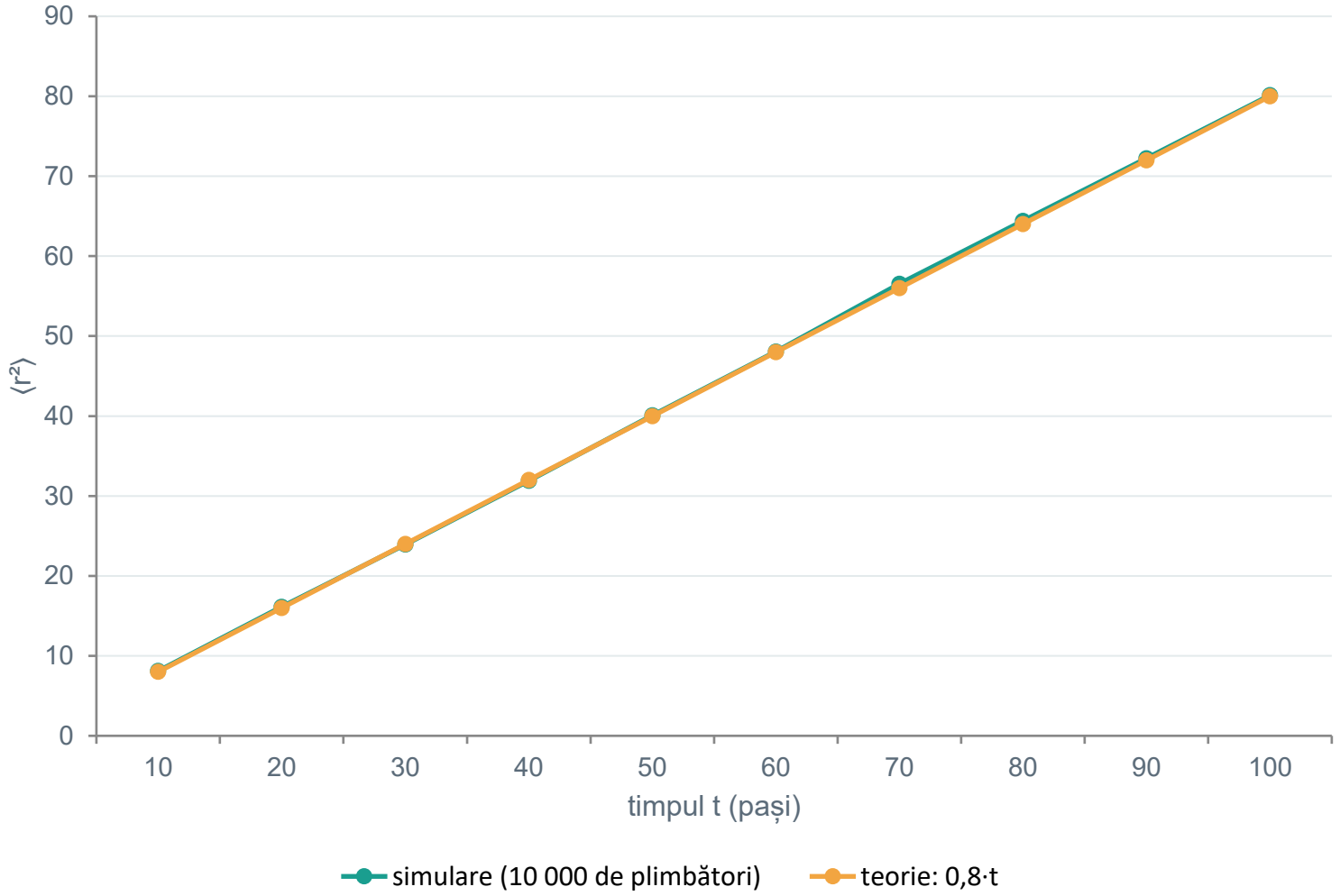
# Deplasarea pătratică medie

$$\langle r^2(t) \rangle = 4\,D\,t \qquad (\hat{\imath}n\ 2D)$$

| | |
|---|---|
| $\langle r^2 \rangle$ | media pătratului distanței față de punctul de plecare |
| $D$ | **coeficientul de difuzie**; aici fiecare pas are $E[\Delta x^2] = E[\Delta y^2] = 2/5$, deci $D = 1/5$ |
| $d$ *dim.* | în general $\langle r^2 \rangle = 2\,d\,D\,t$; în 1D, $\mathrm{Var}(X_n) = n$ (plimbarea simplă) |

Pentru modelul cu 5 mutări: $\langle r^2(t) \rangle = 0{,}8 \cdot t$. La $t = 100$ ne așteptăm la $\langle r^2 \rangle \approx 80$, adică o distanță tipică $\sqrt{80} \approx 8{,}9$.

# $\langle r^2 \rangle$ crește liniar cu timpul




- Punctele simulate stau **pe dreaptă**
- t = 100: $\langle r^2 \rangle$ = **80,1** (teoretic 80)
- Distanța tipică $\sqrt{\langle r^2 \rangle}$ crește ca $\sqrt{t}$
- Legea difuziei, obținută **fără** nicio ecuație de fizică

# Câți plimbători ne trebuie? (t = 100)

| Nr. de plimbători N | ⟨x⟩ | ⟨y⟩ | $\langle r^2 \rangle$ (teoretic 80) |
|---|---|---|---|
| **100** | 0,050 | -0,570 | 92,78 |
| **1 000** | 0,255 | 0,254 | 81,05 |
| **10 000** | -0,046 | -0,031 | 78,90 |
| **100 000** | -0,005 | -0,026 | 80,48 |

*Ca în Cernazean, L3: media se apropie de valoarea teoretică pe măsură ce N crește (eroare ~ 1/√N)*

# De la micro la macro: difuzia

- **Micro**: fiecare particulă face o plimbare aleatoare, fără „plan”
- **Macro**: concentrația se netezește după o lege precisă – **ecuația difuziei** $\partial c/\partial t = D \cdot \nabla^2 c$
- Legea apare **statistic**, din milioane de mișcări independente
- Același tipar în S8 (automate celulare) și S9 (agenți): **reguli locale → comportament global**

**E** **Emergență**

Nicio particulă „nu știe” de difuzie. Legea există doar la nivelul **populației**.

# Difuzia este lentă

$t \sim r^2$

timpul necesar crește cu **pătratul** distanței

**×4**

de 2 ori mai departe → de **4 ori** mai mult timp

**×100**

de 10 ori mai departe → de **100 de ori** mai mult timp

- Un parfum se simte repede în cameră datorită **curenților de aer**, nu difuziei
- În celule, difuzia funcționează bine doar pe distanțe foarte mici

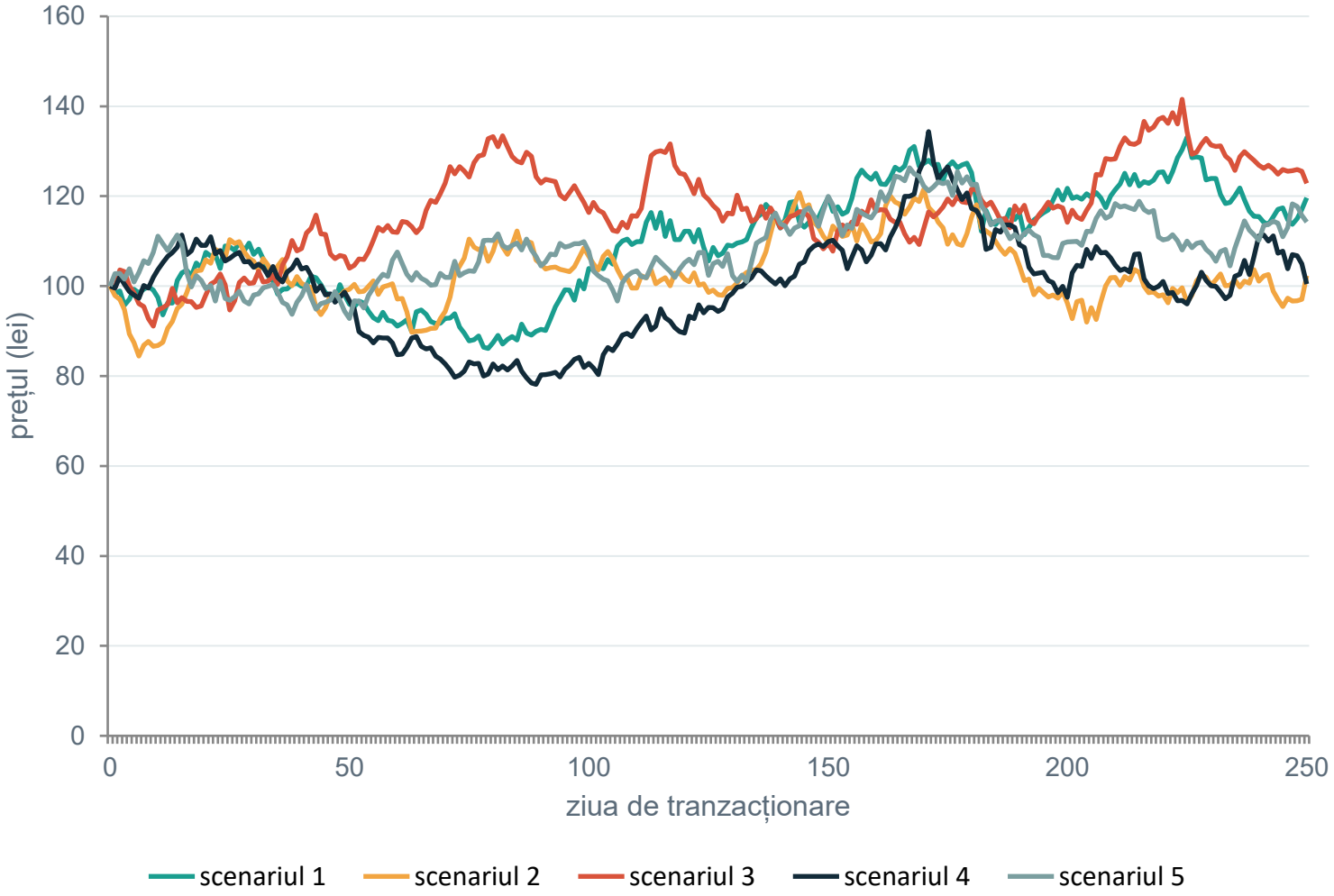
# Mișcarea browniană

## Mișcarea browniană

Limita plimbării aleatoare când pașii devin **foarte mici și foarte deși**: poziția B(t) este continuă în timp, iar **B(t) ~ N(0, t)**.

- Einstein (1905): explicația prin ciocnirile cu moleculele; Perrin (1908) o confirmă experimental
- Traiectorie continuă, dar „zimțată” la orice scară
- Baza modelelor de prețuri și a multor modele de zgomot

# Prețuri: plimbare aleatoare geometrică




- Randamentul zilnic ~ N(0,03%; 2%²)
- $S_{n+1} = S_n \cdot e^{(randament)}$: prețul nu devine negativ
- Aceleași date de pornire, **scenarii foarte diferite** după un an
- Atenție: prețurile reale au **cozi grele** (S2)

# Se întoarce plimbătorul acasă? (Pólya, 1921)

**1D: 100%**

revine sigur în punctul de plecare

**2D: 100%**

revine sigur (dar poate dura foarte mult)

**3D: ≈ 34%**

cu probabilitatea 66% **nu** mai revine niciodată

- „Un om beat își găsește drumul spre casă, dar o pasăre beată se poate pierde pentru totdeauna.” (S. Kakutani)
- Dimensiunea spațiului schimbă calitativ comportamentul

# Unde apare plimbarea aleatoare

### 1 Fizică și chimie

Difuzia moleculelor, căldura, polimerii, mișcarea browniană.

### 2 Biologie

Bacterii care caută hrană, animale, răspândirea genelor.

### 3 Finanțe

Modele de prețuri, risc de portofoliu (S2: Monte Carlo).

### 4 Rețele și grafuri

Eșantionarea rețelelor sociale, crawling, recomandări.

### 5 Algoritmi

Algoritmi randomizați (2-SAT, Papadimitriou 1991), echilibrarea încărcării.

### 6 AI generativ

Modelele de **difuzie** pentru imagini pornesc de la o plimbare aleatoare.

# Plimbări aleatoare pe grafuri

### 1 Eșantionarea rețelelor

Nu putem descărca tot Facebook-ul: pornim o plimbare și colectăm nodurile vizitate.

### 2 Recomandări

Plimbări pe graful utilizatori – produse (de ex. Pinterest Pixie, 2018): ce e „aproape” de ce ți-a plăcut.

### 3 Embeddings de grafuri

DeepWalk (2014), node2vec (2016): secvențele de noduri vizitate sunt tratate ca propoziții.

### 4 PageRank

Plimbarea unui „internaut aleator” pe graful web (ora 3).

# Modelele de difuzie (AI generativ)

Ideea din spatele generatoarelor de imagini (Sohl-Dickstein 2015; Ho et al. 2020):

**1**

**Imagine reală**

$x_0$: o fotografie din setul de antrenare

→

**2**

**Adăugăm zgomot**

la fiecare pas, puțin zgomot gaussian: o plimbare aleatoare în spațiul imaginilor

→

**3**

**Zgomot pur**

după ~1 000 de pași nu mai rămâne nimic din imagine

→

**4**

**Învățăm întoarcerea**

o rețea neurală învață să scoată zgomotul, pas cu pas

→

**5**

**Generare**

pornim de la zgomot nou → **o imagine nouă**

# Plimbarea aleatoare: legile pe scurt

| Model | Media poziției | Dispersia | Distanța tipică |
|---|---|---|---|
| **1D simetrică (±1, p = ½)** | 0 | n | √n |
| **1D cu tendință (p ≠ ½)** | n(2p − 1) | 4np(1 − p) | √(4np(1 − p)) |
| **2D, 5 mutări (Cernazean)** | (0, 0) | $\langle r^2 \rangle = 0{,}8 \cdot t$ | √(0,8·t) |
| **General, d dimensiuni** | 0 (fără tendință) | $\langle r^2 \rangle = 2dDt$ | √(2dDt) |

*Toate provin din același fapt: dispersiile pașilor independenți se adună*

# Cum simulăm corect o plimbare aleatoare

- Alegeți **pasul** (±1, 5 mutări, normal) și verificați media și dispersia unui pas
- Simulați **multe** traiectorii, nu una; vectorizați (o linie = o traiectorie)
- Comparați cu legile cunoscute: $\langle X \rangle = n(2p - 1)$, $\langle r^2 \rangle = 2dDt$
- Raportați media cu **interval de încredere** (S2) și numărul de plimbători
- Aveți grijă la **margini** (pereți, stări absorbante): schimbă rezultatul

✓ **Verificare rapidă**

Dacă $\langle r^2 \rangle$ nu crește liniar în t, pașii nu sunt independenți sau aveți o greșeală de indexare.

# Plimbarea aleatoare este un lanț Markov

- Poziția de **mâine** depinde doar de poziția de **azi** și de o monedă nouă
- Nu contează **cum** am ajuns în poziția curentă: drumul trecut nu mai contează
- Aceasta este **proprietatea Markov** – subiectul orei 2
- Plimbarea pe un graf finit (4 colțuri de stradă, pagini web) → **matrice de tranziție**

**Legătura**

Ora 2 generalizează: orice sistem cu stări și reguli de trecere aleatoare care depind doar de starea curentă.

# Verificare

? O plimbare 1D face **100** de pași, cu probabilitatea **p = 0,6** de a urca. Care este poziția **medie** după 100 de pași?

A 0

B 10

C 20

D 60

# Verificare

? În simularea 2D, după t = 100 de pași am măsurat $\langle r^2 \rangle \approx 80$. Cât estimăm pentru $\langle r^2 \rangle$ la **t = 400**?

A 160

B 320

C 1 280

D 6 400

# Verificare

? Un joc corect (p = ½): pornim cu **30** de lei, pariem câte 1 leu și ne oprim la **0** sau la **100** de lei. Care este probabilitatea să ajungem la 100?

A 0,03

B 0,3

C 0,5

D 0,7

# Rezumatul orei 1

- ✓ Un **proces stocastic** este un șir de variabile aleatoare în timp; o rulare = o traiectorie
- ✓ Plimbarea simplă: $E[X_n] = 0$, $\sigma = \sqrt{n}$; cu tendință: $E[X_n] = n(2p - 1)$
- ✓ Ruina jucătorului: $P = k/N$ în jocul corect; tendința mică + timp lung = pierdere sigură
- ✓ În 2D: $\langle r^2 \rangle = 4Dt$ → difuzia; legea apare din reguli locale (emergență)
- ✓ Aplicații: difuzie, prețuri, grafuri, **modele de difuzie** în AI generativ

Ora 2

# Lanțuri Markov

1. Proprietatea Markov: viitorul depinde doar de prezent
2. Graf, matrice de tranziție, distribuția după n pași
3. Distribuția staționară și comportamentul pe termen lung
4. Clasificarea stărilor; lanțuri absorbante; estimarea din date

# Unde am rămas

- ✓ Plimbarea aleatoare: poziția de mâine = poziția de azi + un pas aleator
- ✓ Trecutul **nu contează**, dacă știm poziția curentă
- ✓ Legile medii apar din multe traiectorii ($\sqrt{n}$, $\langle r^2 \rangle = 4Dt$)
- ✓ Acum: **orice** sistem cu stări discrete și treceri aleatoare

# Proprietatea Markov

## Proprietatea Markov (lipsa memoriei)

$P(X_{n+1} = j \mid X_0, X_1, \ldots, X_{n-1}, \mathbf{X_n = i}) = P(X_{n+1} = j \mid \mathbf{X_n = i}) = \mathbf{P_{ij}}$. „Mâine” depinde doar de „azi”; „ieri” nu aduce informație în plus.

*Holban, cap. 5.1*

- **Omogenitate**: $P_{ij}$ nu depinde de n – aceeași regulă la fiecare pas
- Un **lanț Markov** = un proces cu stări discrete, timp discret și proprietatea Markov
- Numele: Andrei Markov (1856–1922), matematician rus

# Are sistemul proprietatea Markov?

## Da (cu stări bine alese)

- Poziția unei plimbări aleatoare
- Numărul de biciclete dintr-o stație (S1)
- Starea unui server: OK / degradat / căzut
- Poziția pe tabla de „Nu te supăra, frate”

## Nu direct

- Vremea reală (depinde de mai multe zile, de anotimp)
- Un text: cuvântul următor depinde de tot contextul
- Durata rămasă a unei piese cu **uzură** (depinde de vârstă)
- Comportamentul unui utilizator care își amintește

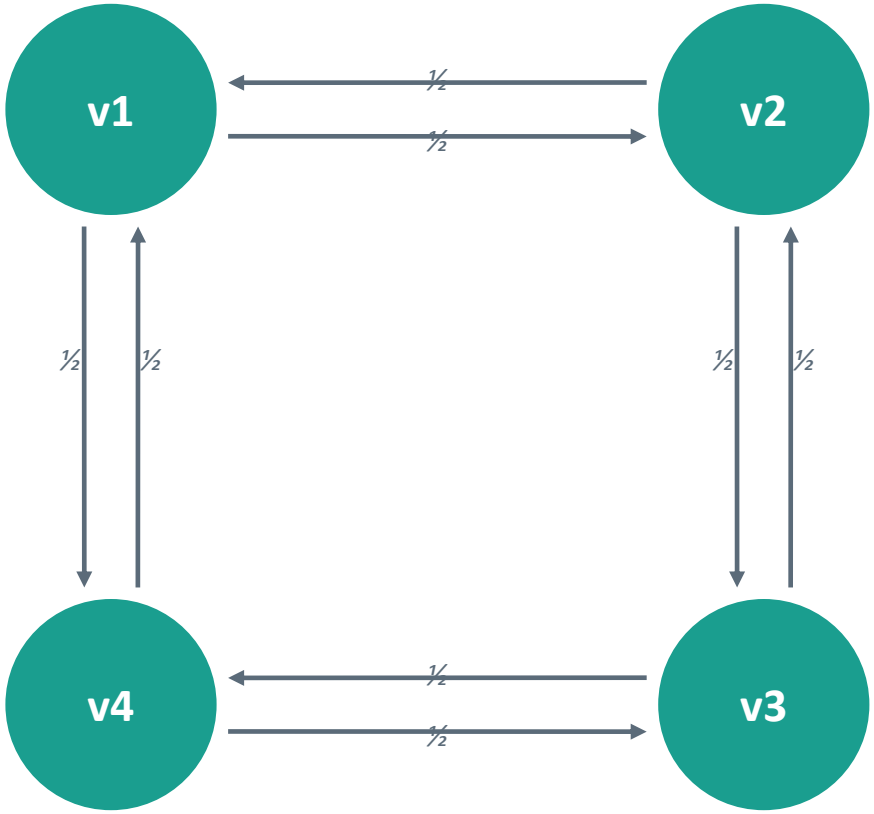
# Trucul: extinderea stării

- Dacă mâine depinde de **azi și ieri**, luăm ca stare **perechea** (ieri, azi)
- Vremea cu 2 stări și memorie de 2 zile → lanț Markov cu **4** stări
- Piesa cu uzură: starea = (funcțională, **vârsta**)
- Costul: numărul de stări crește repede ($k^m$ pentru memorie de m pași)

**Regula de aur**

Starea trebuie să conțină **tot ce e necesar** pentru a prezice pasul următor – și nimic în plus.

# Pietonul aleator (Holban, exemplul 1)




Un oraș cu 4 colțuri de stradă; pietonul pornește din **v1**

La fiecare pas aruncă o monedă: cap → sens orar, pajură → sens trigonometric

$X_n$ = colțul la momentul n; stările: {1, 2, 3, 4}

$P(X_0 = 1) = 1;\ P(X_1 = 2) = P(X_1 = 4) = ½$

# Matricea de tranziție a pietonului

| din \ în | v1 | v2 | v3 | v4 |
|---|---|---|---|---|
| **v1** | 0 | 0,5 | 0 | 0,5 |
| **v2** | 0,5 | 0 | 0,5 | 0 |
| **v3** | 0 | 0,5 | 0 | 0,5 |
| **v4** | 0,5 | 0 | 0,5 | 0 |

*Rândul i = de unde plecăm; coloana j = unde ajungem. Fiecare rând are suma 1.*

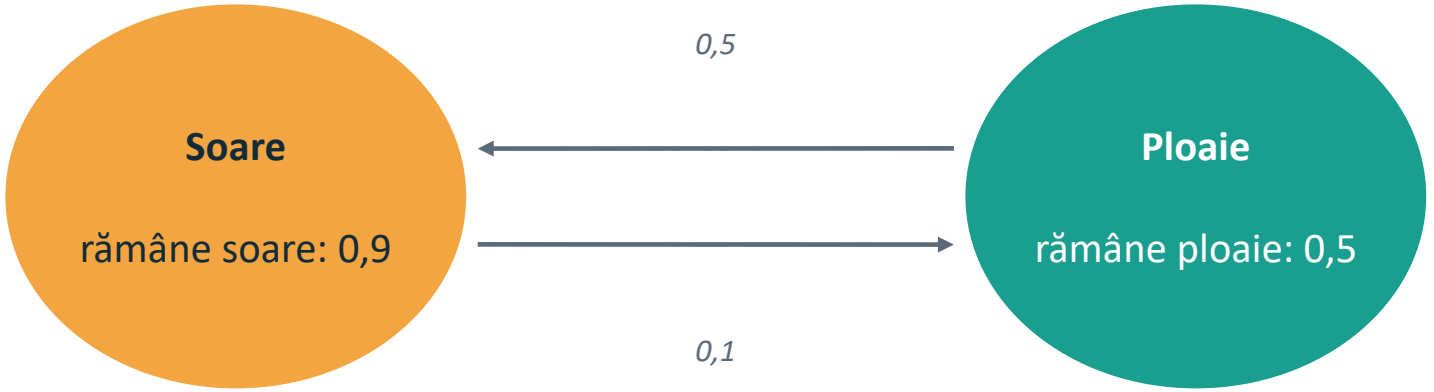
# Matricea de tranziție

## Matrice de tranziție (stocastică)

P = $(P_{ij})$, cu $\mathbf{P_{ij} \geq 0}$ și $\mathbf{\Sigma_j P_{ij} = 1}$ pentru fiecare rând i. Împreună cu distribuția inițială $\boldsymbol{\pi^{(0)}}$, descrie complet lanțul.

- Graful și matricea sunt **două reprezentări** ale aceluiași model
- Săgețile lipsă din graf = zerouri în matrice
- Greșeala clasică: sumele pe **coloane** în loc de rânduri

# Vremea cu două stări (Holban, exemplul 3a)




$$P = \begin{pmatrix} 0{,}9 & 0{,}1 \\ 0{,}5 & 0{,}5 \end{pmatrix}$$

rânduri: soare, ploaie

*O zi însorită e urmată de o zi însorită în 90% din cazuri; o zi ploioasă, de ploaie în 50%*

# Distribuția după un pas

$$\pi^{(1)} = \pi^{(0)} \cdot P$$

$\pi^{(0)}$ vectorul-linie al probabilităților de pornire; azi e soare: $\pi^{(0)} = (1, 0)$

$\pi^{(1)}$ $(1, 0) \cdot P = (0{,}9;\ 0{,}1)$: mâine, 90% soare

$\pi^{(2)}$ $(0{,}9;\ 0{,}1) \cdot P = (0{,}9 \cdot 0{,}9 + 0{,}1 \cdot 0{,}5;\ \ldots) =$ **(0,86; 0,14)**

Fiecare componentă: $\pi_j^{(n+1)} = \Sigma_i\, \pi_i^{(n)} \cdot P_{ij}$ – „șansa de a fi în i” × „șansa de a trece din i în j”, adunate peste toate i.

# Distribuția după n pași

$$\pi^{(n)} = \pi^{(0)} \cdot P^n$$

| | |
|---|---|
| $P^n$ | puterea n a matricei; $(P^n)_{ij} = P(X_n = j \mid X_0 = i)$ |
| $P^{m+n}$ | $= P^m \cdot P^n$ (Chapman–Kolmogorov): drumul de m + n pași trece printr-o stare intermediară |
| *calcul* | k stări, n pași: n înmulțiri vector × matrice, fiecare cu $k^2$ operații |

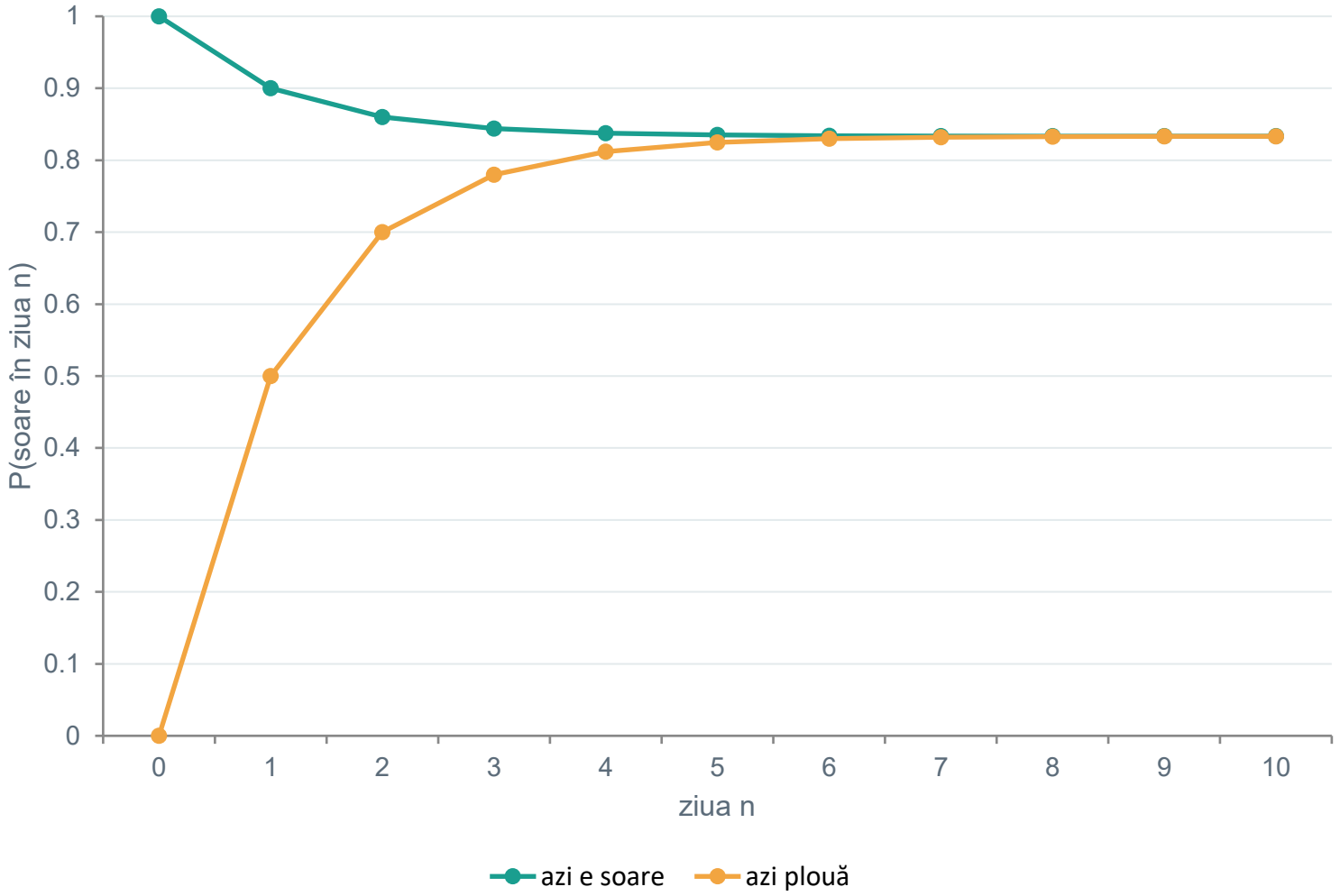
Holban, teorema 1: întreaga evoluție a lanțului se reduce la **înmulțiri de matrice**.

# Vremea, zi cu zi

| Ziua n | P(soare) dacă azi e soare | P(soare) dacă azi plouă |
|---|---|---|
| **0** | 1,0000 | 0,0000 |
| **1** | 0,9000 | 0,5000 |
| **2** | 0,8600 | 0,7000 |
| **3** | 0,8440 | 0,7800 |
| **4** | 0,8376 | 0,8120 |
| **5** | 0,8350 | 0,8248 |
| **10** | 0,8334 | 0,8332 |
| **20** | 0,8333 | 0,8333 |

*Valorile $(P^n)_{ss}$ și $(P^n)_{ps}$, calculate cu NumPy*

# Lanțul își uită punctul de plecare




- Ambele curbe tind spre **0,8333 = 5/6**
- Diferența se înmulțește cu 0,4 în fiecare zi
- Pe termen lung: **5 zile din 6** sunt însorite
- Această limită = **distribuția staționară**

# $P^n$ în NumPy

```
import numpy as np

P = np.array([[0.9, 0.1],      # soare
              [0.5, 0.5]])     # ploaie

pi0 = np.array([1.0, 0.0])     # azi: soare
for n in [1, 2, 5, 10]:
    Pn = np.linalg.matrix_power(P, n)
    print(n, pi0 @ Pn)

# 1 [0.9  0.1 ]
# 2 [0.86 0.14]
# 5 [0.835 0.165]
# 10 [0.8334 0.1666]
```

- **@** = înmulțire de matrice în Python
- **matrix_power** calculează $P^n$
- Vectorul de probabilități este un vector-**linie**
- Verificați mereu: suma componentelor = 1

# Verificare

? Cu modelul de vreme (soare → soare 0,9; ploaie → soare 0,5), **azi plouă**. Care este probabilitatea să fie **soare poimâine**?

A 0,5

B 0,7

C 0,86

D 0,9

# Vremea cu trei stări (Holban, exemplul 3b)

| azi \ mâine | Ploaie | Înnorat | Soare |
|---|---|---|---|
| **Ploaie** | 0,4 | 0,3 | 0,3 |
| **Înnorat** | 0,2 | 0,6 | 0,2 |
| **Soare** | 0,1 | 0,1 | 0,8 |

*Modelul clasic al lui Rabiner (1989); distribuția staționară: (0,182; 0,273; 0,545)*

# Probabilitatea unei secvențe

$$P(x_0, x_1, \ldots, x_n) = \pi^{(0)}(x_0) \cdot P_{x0x1} \cdot P_{x1x2} \cdots P_{xn-1xn}$$

***întrebarea*** azi e soare; care e probabilitatea secvenței S S S P P S Î S?

***produsul*** $1 \cdot P(S|S) \cdot P(S|S) \cdot P(P|S) \cdot P(P|P) \cdot P(S|P) \cdot P(Î|S) \cdot P(S|Î)$

***rezultat*** $1 \cdot 0{,}8 \cdot 0{,}8 \cdot 0{,}1 \cdot 0{,}4 \cdot 0{,}3 \cdot 0{,}1 \cdot 0{,}2 =$ **$1{,}536 \cdot 10^{-4}$**

Datorită proprietății Markov, probabilitatea unui drum este un **produs** de probabilități de tranziție.

# Simularea unui lanț Markov

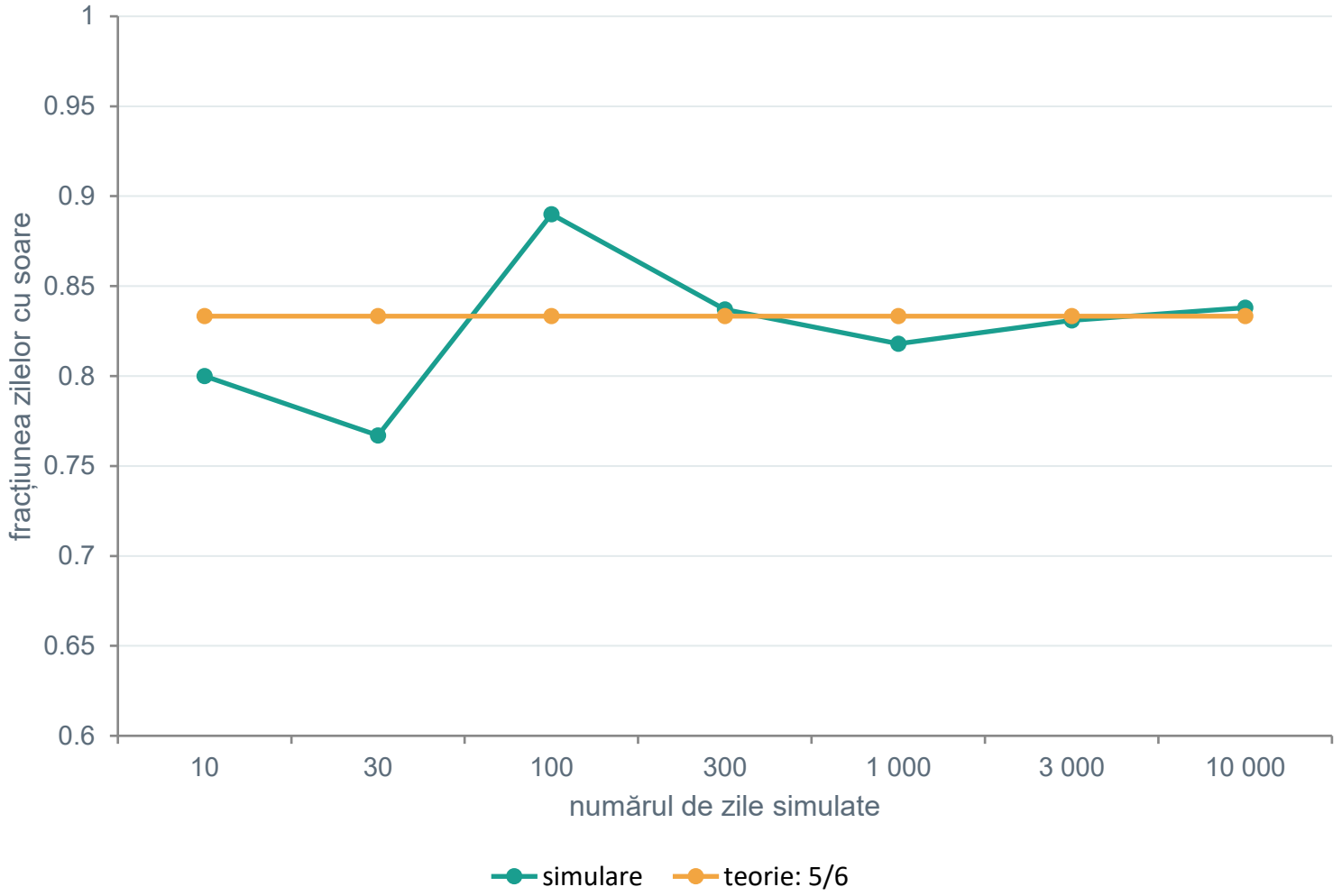
```
import numpy as np
rng = np.random.default_rng(10)
P = np.array([[0.9, 0.1], [0.5, 0.5]])

x, days, sunny = 0, 10_000, []
for _ in range(days):
    # starea următoare, din rândul stării curente
    x = rng.choice(2, p=P[x])
    sunny.append(x == 0)

print(np.mean(sunny))   # fracția zilelor cu soare
```

- La fiecare pas: o variabilă **discretă** cu probabilitățile din rândul P[x]
- Exact transformarea inversă din S2
- Rezultat: **0,8378** (teoretic 0,8333)
- Un singur drum lung → media în timp

# Fracțiunea zilelor cu soare, pe un singur drum




- La început: fluctuații mari
- Pe termen lung: media în timp → **π(soare)**
- Convergența e mai lentă decât la eșantioane independente
- Motiv: zilele consecutive sunt **corelate**

# Două drumuri spre comportamentul pe termen lung

## Calcul (analitic / numeric)

- $\pi^{(n)} = \pi^{(0)}P^n$ pentru n mare
- sau rezolvăm direct $\pi = \pi P$
- exact, rapid pentru zeci – mii de stări
- cere matricea completă

vs

## Simulare

- un drum lung sau multe drumuri scurte
- fracțiunea de timp în fiecare stare
- aproximativ, cu eroare ~ $1/\sqrt{N}$
- merge și când matricea e uriașă sau necunoscută

# Distribuția staționară

## Distribuție staționară

Un vector de probabilități $\boldsymbol{\pi}$ cu $\boldsymbol{\pi} = \boldsymbol{\pi} \cdot \mathbf{P}$ și $\boldsymbol{\Sigma}\, \boldsymbol{\pi}_i = \mathbf{1}$. Dacă pornim din π, rămânem în π la fiecare pas.

*Holban, cap. 5.5*

- $\boldsymbol{\pi}_i$ = fracțiunea de timp petrecută în starea i, pe termen lung
- Este un vector propriu la stânga al lui P, pentru valoarea proprie 1
- Sistemul ajunge în „regim permanent” după un regim **tranzitoriu**

# Două stări: o formulă de reținut

$$\pi_1 = b / (a + b), \quad \pi_2 = a / (a + b)$$

| | |
|---|---|
| ***a, b*** | a = P(1 → 2), b = P(2 → 1): probabilitățile de „plecare” din fiecare stare |
| ***echilibru*** | fluxul 1 → 2 egal cu fluxul 2 → 1: $\pi_1 \cdot a = \pi_2 \cdot b$ |
| ***vremea*** | a = 0,1, b = 0,5: π(soare) = 0,5 / 0,6 = **5/6 ≈ 0,833** |

Intuiția echilibrului: pe termen lung, de câte ori „ieși” dintr-o stare, de atâtea ori „intri” în ea.

# Trei stări: exemplul A (Holban)

$$\pi = \pi Q, \quad Q = [\,0 \;\; 1 \;\; 0\,;\;\; ¼ \;\; ¼ \;\; ½\,;\;\; 0 \;\; ½ \;\; ½\,]$$

***ecuațiile*** $\pi_1 = ¼\pi_2; \quad \pi_2 = \pi_1 + ¼\pi_2 + ½\pi_3; \quad \pi_3 = ½\pi_2 + ½\pi_3$

***din ele*** $\pi_2 = 4\pi_1$ și $\pi_3 = \pi_2 = 4\pi_1$

***normare*** $\pi_1 + 4\pi_1 + 4\pi_1 = 1 \;\rightarrow\; \mathbf{\pi = (1/9, 4/9, 4/9)}$

Una dintre ecuațiile $\pi = \pi Q$ este mereu redundantă; o înlocuim cu condiția de normare $\Sigma\pi_i = 1$.

# Distribuția staționară în NumPy

```
import numpy as np
Q = np.array([[0,    1,    0  ],
              [0.25, 0.25, 0.5],
              [0,    0.5,  0.5]])
k = len(Q)

# (Q^T − I) π^T = 0  și  Σ π = 1
A = np.vstack([Q.T - np.eye(k), np.ones(k)])
b = np.r_[np.zeros(k), 1]
pi = np.linalg.lstsq(A, b, rcond=None)[0]
print(pi)   # [0.111 0.444 0.444]

# formula din Holban: π = 1 · (Q + E − I)^-1
print(np.ones(k) @ np.linalg.inv(Q + 1 - np.eye(k)))
```

- Adăugăm ecuația Σπ = 1 la sistemul π(Q − I) = 0
- **lstsq** rezolvă sistemul supradeterminat
- Formula lui Holban dă același rezultat
- Pentru milioane de stări: iterăm π ← πQ (ora 3)

# Ce ne spune π: indici de performanță

| Indice | Formula | Exemplul A, starea 3 (pas = 50 u.t.) |
|---|---|---|
| **Fracțiunea de timp în starea i** | $\pi_i$ | 4/9 ≈ 44% din timp |
| **Gradul de utilizare (dacă i = „ocupat”)** | $\pi_i$ | 44% |
| **Timpul mediu de ședere în i** | 1 / (1 – $P_{ii}$) pași | 1/(1 – ½) = 2 pași = **100 u.t.** |
| **Timpul mediu de revenire în i** | 1 / $\pi_i$ pași | 9/4 pași = **112,5 u.t.** |

*Timpul de ședere folosește $P_{ii}$ (probabilitatea de a rămâne), nu $\pi_i$*

# Clasificarea stărilor

### 1 Accesibilă

j e accesibilă din i dacă există un drum i → … → j cu probabilitate > 0.

### 2 Ireductibil

Orice stare e accesibilă din oricare alta: lanțul nu se „rupe” în bucăți.

### 3 Recurentă / tranzitorie

Recurentă: revii sigur. Tranzitorie: la un moment dat pleci definitiv.

### 4 Absorbantă

$P_{ii} = 1$: odată intrat, nu mai ieși (ruina, „cumpărat”, „defect definitiv”).

### 5 Periodică

Revii doar la multipli de d > 1 pași (pietonul: doar la pași pari).

### 6 Ergodic

Finit, ireductibil și aperiodic: are o π unică, atinsă din orice start.

# Periodic vs aperiodic

## Pietonul (perioada 2)

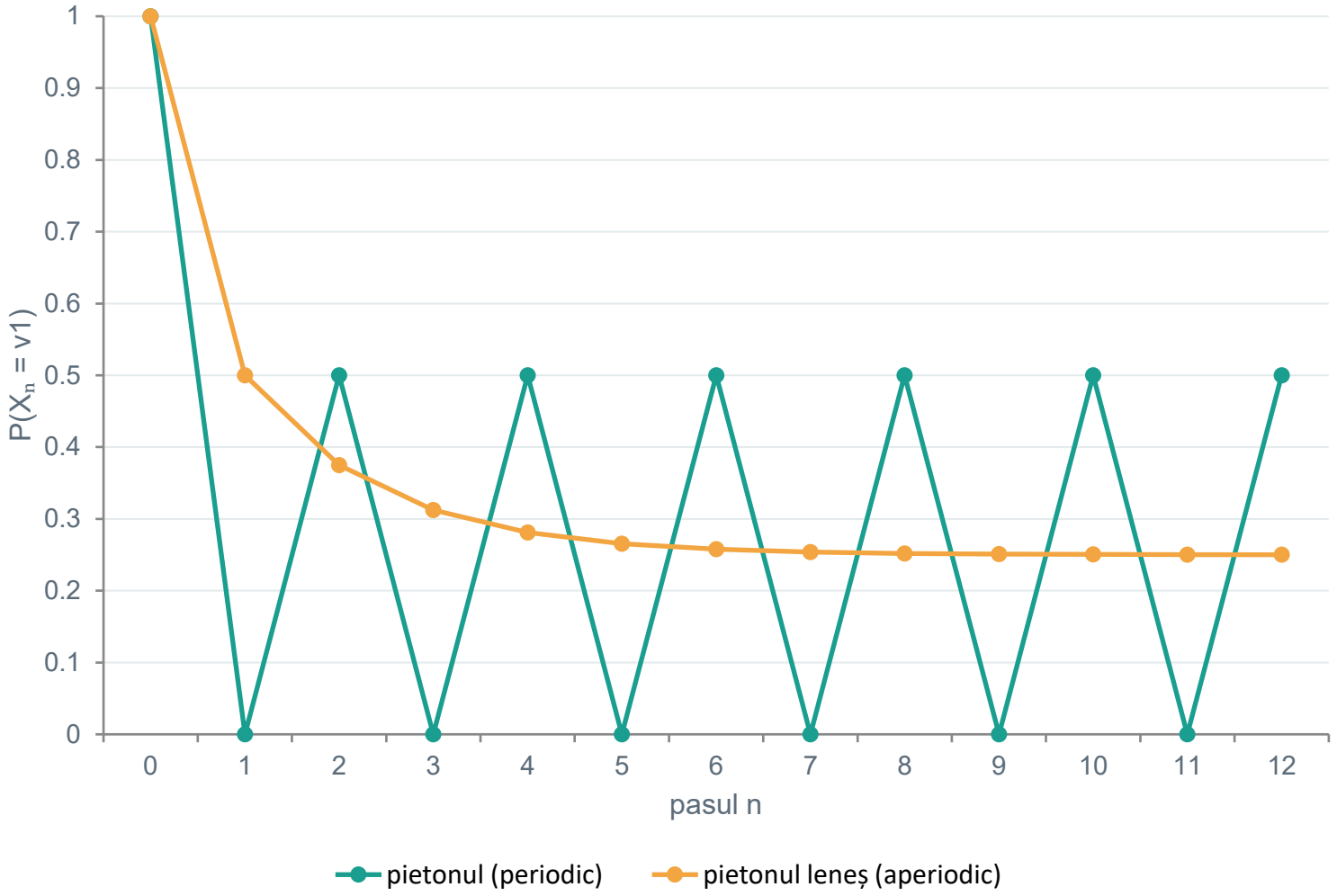
- Din v1 ajunge în v2 sau v4, apoi în v1 sau v3
- La pașii **pari**: doar v1 sau v3
- $\pi^{(n)}$ **oscilează** și nu converge
- Dar $\pi = (¼, ¼, ¼, ¼)$ există: fracțiunea de timp

vs

## Pietonul „leneș”

- Cu probabilitatea ½ rămâne pe loc
- Apare $P_{ii} > 0$ → lanțul devine **aperiodic**
- $\pi^{(n)} \to (¼, ¼, ¼, ¼)$ din orice start
- Truc folosit des în algoritmi (lazy random walk)

# P(pietonul este în v1 la pasul n)




- Periodic: 1, 0, ½, 0, ½, 0, … – **nu converge**
- Leneș: 1; 0,5; 0,375; … → **0,25**
- Media în timp e ¼ în ambele cazuri
- Convergența lui $\pi^{(n)}$ cere **aperiodicitate**

# Teorema ergodică (fără demonstrație)

- Dacă lanțul este **finit, ireductibil și aperiodic**:
- 1. există o **unică** distribuție staționară $\pi$, cu toate $\pi_i > 0$
- 2. $\pi^{(n)} \to \pi$, **indiferent** de punctul de plecare
- 3. fracțiunea de timp petrecută în i, pe un singur drum lung → $\boldsymbol{\pi_i}$
- Punctul 3 justifică simularea: un drum lung ≈ media pe multe drumuri

**H** **Holban**

Un sistem pentru care limitele probabilităților există se numește **ergodic**, iar procesul – proces ergodic.

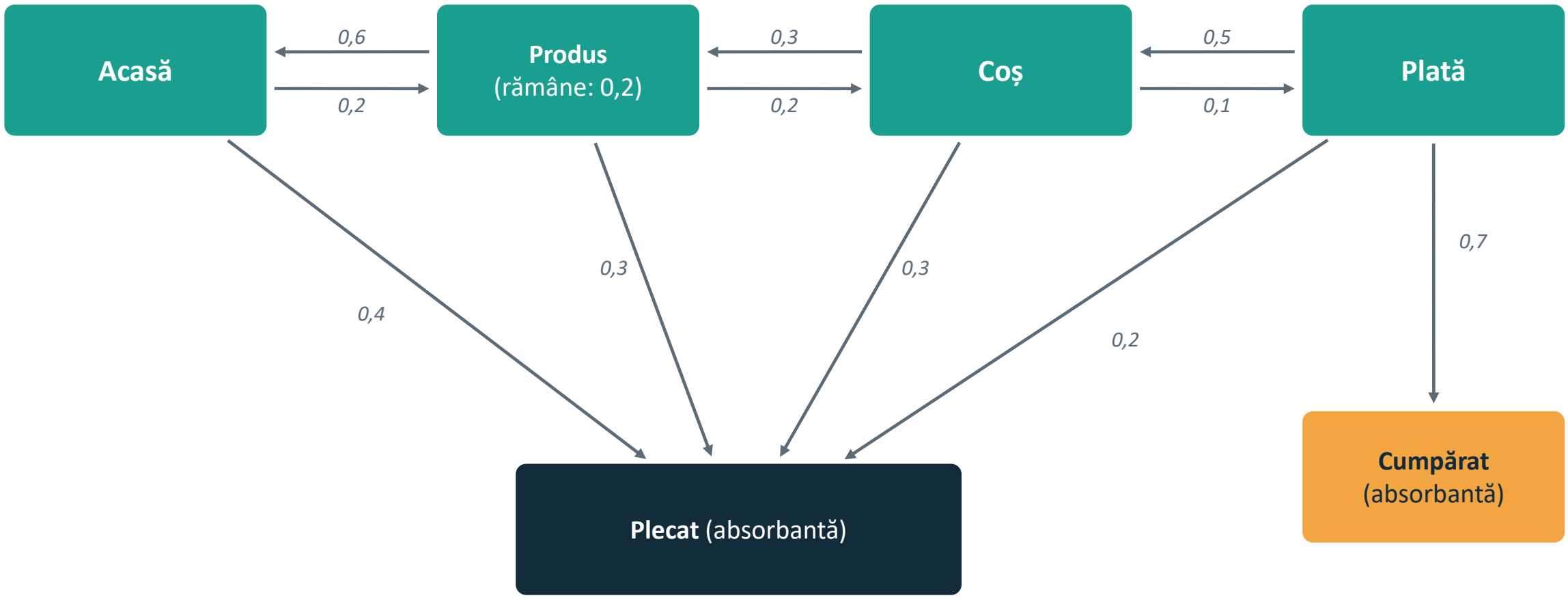
# Lanțuri absorbante

## Lanț absorbant

Are cel puțin o stare **absorbantă** ($P_{ii} = 1$) și din orice stare se poate ajunge într-una absorbantă. Celelalte stări sunt **tranzitorii**.

- Întrebări tipice: **în ce stare** absorbantă ajungem și **după cât timp**?
- Exemple: ruina jucătorului, un client pe un site, un pacient, o piesă până la defect
- Aici nu ne interesează $\pi$, ci **probabilitățile de absorbție**

# Exemplu: vizita pe un magazin online



# Matricea pâlniei: stări tranzitorii și absorbante

| din \ în | Acasă | Produs | Coș | Plată | Cumpărat | Plecat |
|---|---|---|---|---|---|---|
| **Acasă** | 0 | 0,6 | 0 | 0 | 0 | 0,4 |
| **Produs** | 0,2 | 0,2 | 0,3 | 0 | 0 | 0,3 |
| **Coș** | 0 | 0,2 | 0 | 0,5 | 0 | 0,3 |
| **Plată** | 0 | 0 | 0,1 | 0 | 0,7 | 0,2 |
| **Cumpărat** | 0 | 0 | 0 | 0 | 1 | 0 |
| **Plecat** | 0 | 0 | 0 | 0 | 0 | 1 |

*Blocul 4 × 4 din stânga sus = Q (tranzitorii → tranzitorii); blocul 4 × 2 din dreapta sus = R (tranzitorii → absorbante)*

# Matricea fundamentală

$$N = (I - Q)^{-1}, \quad B = N \cdot R, \quad t = N \cdot 1$$

| | |
|---|---|
| $N_{ij}$ | numărul mediu de vizite în starea tranzitorie j, pornind din i |
| $B_{ia}$ | probabilitatea de a ajunge în starea absorbantă a, pornind din i |
| $t_i$ | numărul mediu de pași (pagini) până la absorbție |

N = I + Q + Q² + … : adunăm vizitele de la toți pașii. Se calculează cu o singură inversare de matrice.

# Pâlnia magazinului: rezultate

**10,8%**

din vizitele care încep pe **Acasă** se termină cu o cumpărare (simulare: 10,9%)

**2,63**

pagini vizitate în medie, pornind de pe Acasă (simulare: 2,62)

**74,1%**

din vizitatorii ajunși la **Plată** cumpără (de la Coș: 40,6%)

- Întrebare de proiectare: unde merită îmbunătățit site-ul? Modificăm o probabilitate și recalculăm
- Aceleași calcule: fiabilitate (piesă → defect), sănătate, ruina jucătorului

# Estimarea matricei din date

| din \ în (1 000 de zile) | → OK | → Degradat | → Căzut | total |
|---|---|---|---|---|
| **OK** | 872 (0,967) | 23 (0,025) | 7 (0,008) | 902 |
| **Degradat** | 15 (0,224) | 44 (0,657) | 8 (0,119) | 67 |
| **Căzut** | 15 (0,484) | 0 (0,000) | 16 (0,516) | 31 |

*Numărăm tranzițiile și împărțim la totalul pe rând: $\hat{P}_{ij} = n_{ij} / n_i$. Valorile adevărate: rândul Căzut = (0,5; 0; 0,5)*

# Utilajul care se uzează (Holban, exemplul 2)

| După k ani de exploatare | Fără intervenții | Necesită intervenții |
|---|---|---|
| **0** | 1,0000 | 0,0000 |
| **1** | 0,9000 | 0,1000 |
| **2** | 0,8200 | 0,1800 |
| **3** | 0,7560 | 0,2440 |
| **4** | 0,7048 | 0,2952 |
| **5** | 0,6638 | 0,3362 |
| **6** | 0,6311 | 0,3689 |
| **7** | 0,6049 | 0,3951 |
| **8** | 0,5839 | 0,4161 |
| **9** | 0,5671 | 0,4329 |

*Din p(0), p(1), p(2) rezultă Q = [0,9 0,1; 0,1 0,9]; pragul de 40% este depășit după 8 ani; limita este 50%*

# Verificare

? Un lanț cu două stări are P(A → B) = **0,2** și P(B → A) = **0,3**. Ce fracțiune din timp petrece sistemul în starea **A**, pe termen lung?

A 0,2

B 0,4

C 0,6

D 0,8

# Verificare

? Pentru care dintre lanțurile de mai jos distribuția $\pi^{(n)}$ **nu** converge, pentru anumite puncte de plecare?

**A** Vremea cu 2 stări (0,9 / 0,1 / 0,5 / 0,5)

**B** Pietonul pe 4 colțuri (sens orar / trigonometric)

**C** Pietonul leneș (rămâne pe loc cu ½)

**D** Un lanț cu toate $P_{ij} > 0$

# Rezumatul orei 2

- ✓ **Proprietatea Markov**: viitorul depinde doar de starea curentă; la nevoie extindem starea
- ✓ Lanț = stări + **matrice de tranziție** (rânduri cu suma 1) + distribuție inițială
- ✓ $\boldsymbol{\pi^{(n)} = \pi^{(0)} P^n}$; probabilitatea unui drum = produs de $P_{ij}$
- ✓ **Distribuția staționară** $\pi = \pi P$: fracțiunea de timp; două stări: $\pi_1 = b/(a + b)$
- ✓ Ergodic (ireductibil + aperiodic) → $\pi$ unică, atinsă din orice start
- ✓ Lanțuri **absorbante**: $N = (I - Q)^{-1}$; matricea se **estimează din date**

Ora 3

# Lanțuri Markov în informatică

1. PageRank: internautul aleator
2. Limbajul: de la Markov (1913) la n-grame și LLM-uri
3. Fiabilitate și disponibilitate
4. MCMC (Metropolis) și teaser: procese de decizie Markov

# Lanțuri Markov peste tot

### 1 Căutare web

PageRank = distribuția staționară a unui internaut aleator.

### 2 Text

Sugestiile tastaturii, generatoare de text, modele de limbaj clasice.

### 3 Fiabilitate

Servere, rețele, piese: stări OK / defect, disponibilitate.

### 4 Performanță

Cozi și rețele de cozi (săptămâna 4) sunt lanțuri Markov.

### 5 Statistică și ML

MCMC: eșantionare din distribuții complicate (Bayes).

### 6 Învățare prin întărire

MDP: lanț Markov + acțiuni + recompense.

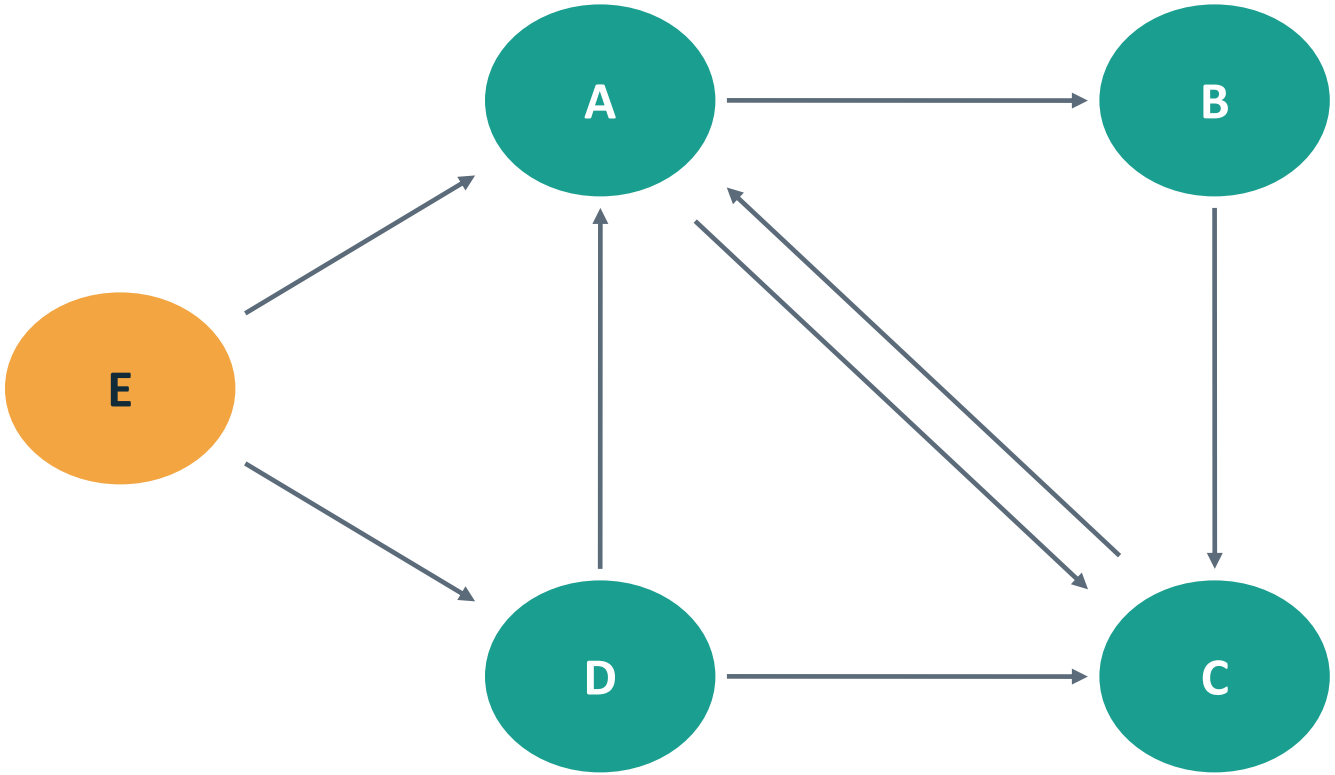
# PageRank: ideea

## Internautul aleator (Brin & Page, 1998)

Un utilizator dă click la întâmplare pe unul din linkurile paginii curente; din când în când se plictisește și sare la o pagină **oarecare**. **PageRank-ul unei pagini = fracțiunea de timp petrecută pe ea** pe termen lung.

- Stările lanțului: paginile web; tranzițiile: linkurile
- Importanța = **distribuția staționară** a acestui lanț Markov
- O pagină e importantă dacă e legată de pagini importante

# Un mini-web cu 5 pagini




A → B, C · B → C · C → A

D → A, C · E → A, D

Un click alege uniform un link: din A, B sau C cu câte ½

*Nicio pagină nu trimite spre E; A și C primesc câte 3 linkuri*

# Două probleme și o soluție: teleportarea

- **Capcane**: A → B → C → A formează un ciclu din care internautul nu mai iese; D și E ajung la PageRank **0**
- **Fundături**: pagini fără linkuri (un PDF) – rândul matricei ar avea suma 0
- **Soluția**: cu probabilitatea **d = 0,85** urmăm un link, cu **0,15** sărim la o pagină oarecare
- Lanțul devine **ireductibil și aperiodic** → π unică (teorema ergodică)

**Legătura cu ora 2**

Teleportarea transformă orice graf web într-un lanț **ergodic**.

# Formula PageRank

$$G = d \cdot M + (1 - d) / n \cdot E, \qquad \pi = \pi \cdot G$$

| | |
|---|---|
| $M$ | matricea linkurilor: $M_{ij}$ = 1 / (nr. de linkuri din i), dacă i → j |
| $d$ | **factorul de amortizare**, de obicei 0,85 (Brin & Page, 1998) |
| $E$ | matricea cu toate elementele 1: teleportarea uniformă pe cele n pagini |

G este „matricea Google”: stocastică, cu toate elementele > 0. PageRank = vectorul π cu π = πG și Σπ = 1.

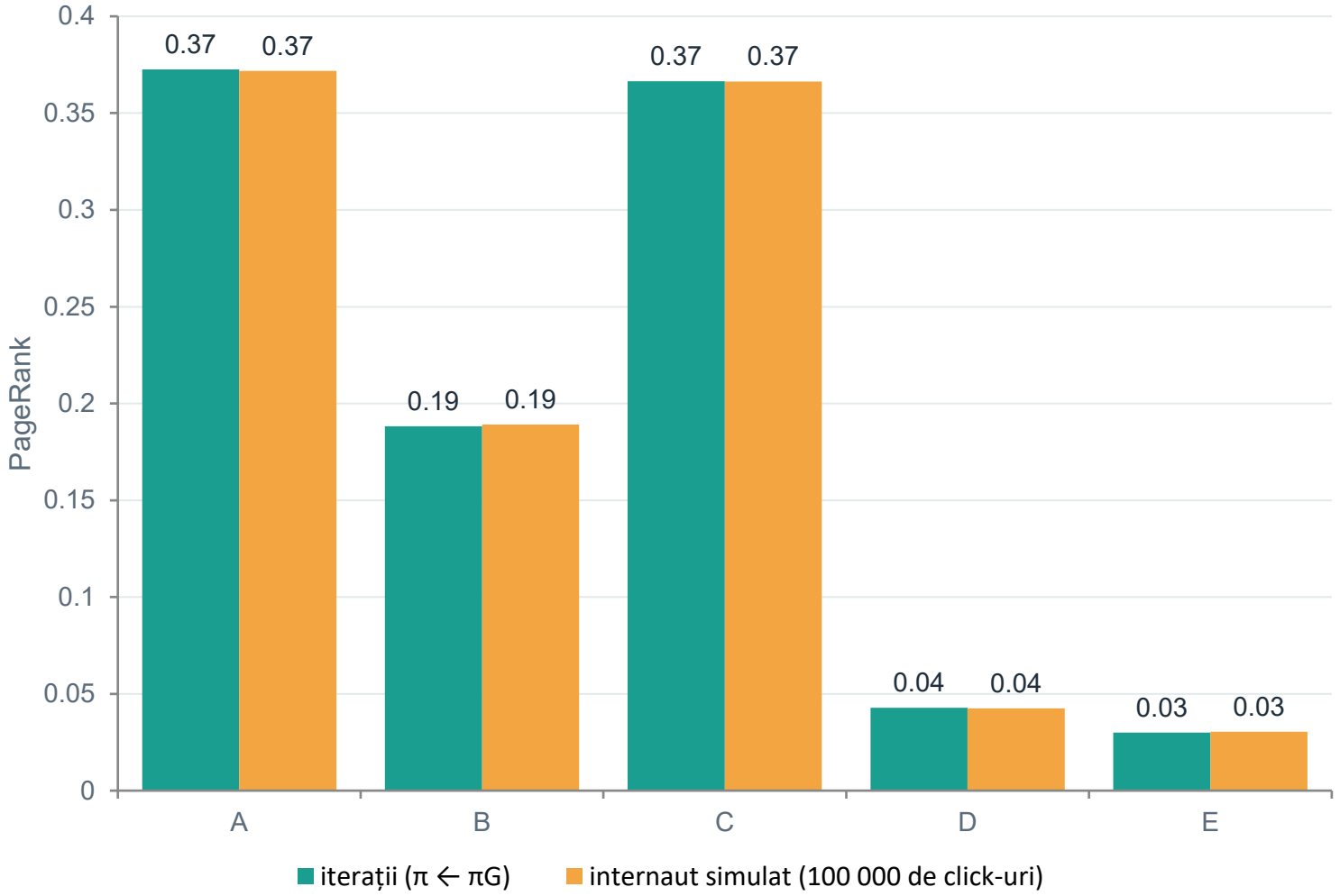
# PageRank prin iterații (power iteration)

```
import numpy as np
pages = ['A', 'B', 'C', 'D', 'E']
links = {'A': ['B', 'C'], 'B': ['C'], 'C': ['A'],
         'D': ['A', 'C'], 'E': ['A', 'D']}
n, idx = 5, {p: i for i, p in enumerate(pages)}
M = np.zeros((n, n))
for a, outs in links.items():
    for b in outs:
        M[idx[a], idx[b]] = 1 / len(outs)

d = 0.85
G = d * M + (1 - d) / n
pr = np.full(n, 1 / n)          # start uniform
for _ in range(60):
    pr = pr @ G                  # π ← π G
print(pr.round(4))
```

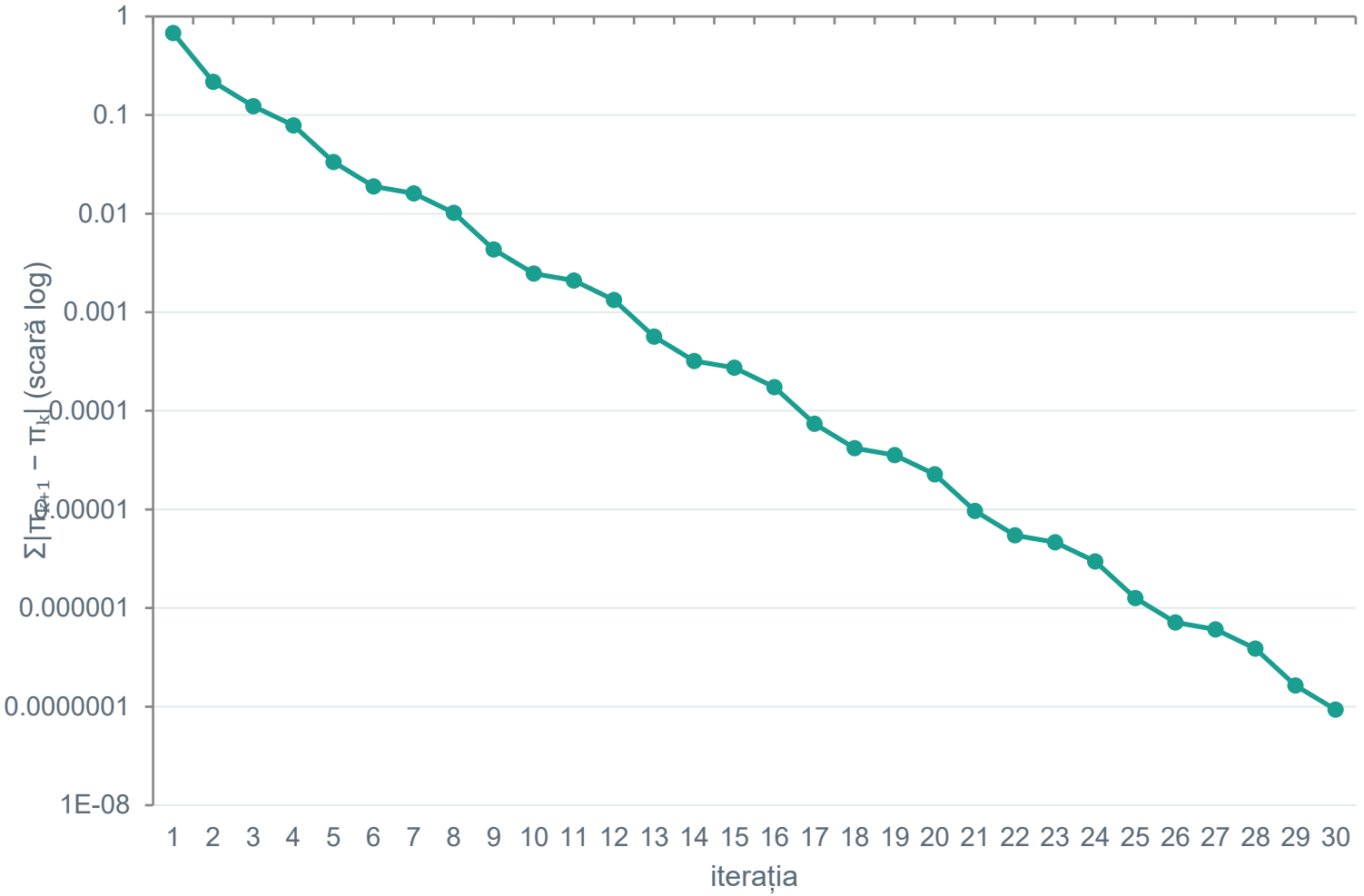
- Nu rezolvăm sistemul: **iterăm** $\pi \leftarrow \pi G$
- Fiecare iterație = un pas al tuturor internauților
- Pentru web-ul real, M e **rară** (sparse): ~10 linkuri pe pagină
- Rezultat: 0,372; 0,188; 0,366; 0,043; 0,030

# PageRank-ul celor 5 pagini




- **A** (0,372) puțin peste **C** (0,366): ambele au 3 linkuri, dar C își dă **tot** votul lui A
- **B** (0,188): un singur link, dar de la A, pagina cea mai importantă
- **E**: doar teleportare: 0,15/5 = **0,03**
- Simularea confirmă calculul

# Cât de repede converg iterațiile?




- Sub $10^{-6}$ după **26** de iterații
- Eroarea scade geometric, cu un factor ≤ **d = 0,85**
- Web-ul real: ~50 de iterații sunt suficiente
- d mai mare → rang mai fidel linkurilor, dar convergență mai lentă

# Cu și fără teleportare

| Pagina | Fără teleportare (d = 1) | Cu teleportare (d = 0,85) | Linkuri primite |
|---|---|---|---|
| **A** | 0,400 | 0,372 | 3 |
| **B** | 0,200 | 0,188 | 1 |
| **C** | 0,400 | 0,366 | 3 |
| **D** | 0,000 | 0,043 | 1 |
| **E** | 0,000 | 0,030 | 0 |

*Fără teleportare, internautul rămâne prins în ciclul A–B–C: D și E primesc 0*

# Ce învățăm din PageRank

- Importanța nu se numără, se **moștenește**: un link de la o pagină importantă valorează mai mult
- Rangul = **distribuția staționară** – un calcul global, nu o proprietate locală a paginii
- **Manipulare**: „ferme de linkuri” care își trimit linkuri una alteia → motoarele de căutare au adăugat sute de alte semnale
- Aceeași idee pentru orice graf: articole citate, influență în rețele sociale, TextRank (rezumarea textelor), rețele de proteine

G **Scara reală**

Miliarde de pagini, matrice rară, câteva zeci de iterații – posibile doar prin **iterare**, nu prin rezolvarea sistemului.

Lanțuri Markov și limbajul

# De la Pușkin la modelele de limbaj

1. Markov (1913): vocale și consoane în „Evgheni Oneghin”
2. Modelele cu n-grame
3. Un generator de text în 10 linii
4. n-grame vs modele mari de limbaj (LLM)

# Markov, 1913: primele 20 000 de litere din „Evgheni Oneghin”

**0,128**

P(vocală | litera anterioară e **vocală**)

**0,663**

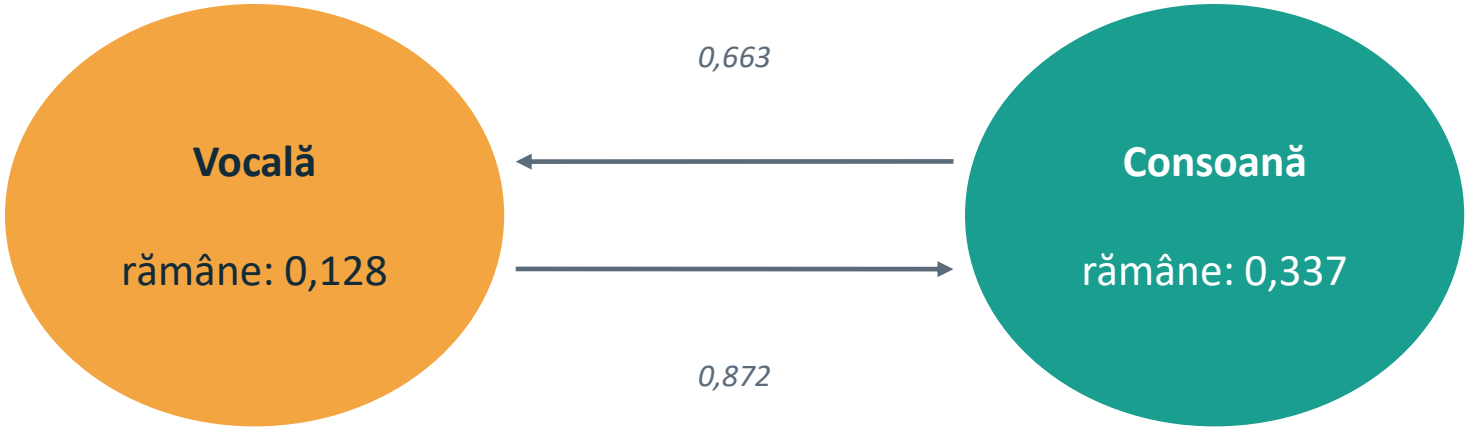
P(vocală | litera anterioară e **consoană**)

**0,432**

fracțiunea de vocale prezisă de lanț: 0,663 / (0,663 + 0,872)

- Literele **nu** sunt independente: după o vocală urmează aproape mereu o consoană
- Markov a numărat manual, ca să arate că legea numerelor mari merge și pentru variabile dependente

# Lanțul lui Markov: vocale și consoane




Formula cu 2 stări:
$\pi(V) = b / (a + b)$
$= 0{,}663 / 1{,}535$
$\approx$
**0,432**

*Aceeași formulă ca la vreme: $a = P(V \to C) = 0{,}872$, $b = P(C \to V) = 0{,}663$*

# Modelul cu n-grame

## Model de limbaj cu n-grame

Probabilitatea cuvântului următor depinde doar de ultimele **n – 1** cuvinte: $P(w_t \mid w_1 \dots w_{t-1}) \approx P(w_t \mid w_{t-n+1} \dots w_{t-1})$, estimată prin **numărare** într-un corpus.

- n = 2 (bigrame): starea = ultimul cuvânt → un lanț Markov pe cuvinte
- Estimare: $\hat{P}(b \mid a)$ = nr(a urmat de b) / nr(a) – ca la jurnalul serverului
- Shannon (1948) a generat astfel texte „aproape englezești”

# Un generator de text cu bigrame

```
import numpy as np, collections
words = corpus.split()          # textul, cuvânt cu cuvânt

# pentru fiecare cuvânt: lista cuvintelor care îl urmează
nxt = collections.defaultdict(list)
for a, b in zip(words[:-1], words[1:]):
    nxt[a].append(b)

rng = np.random.default_rng(3)
w, sent = '.', []
while True:
    w = rng.choice(nxt[w])       # pasul lanțului Markov
    if w == '.':
        break
    sent.append(w)
print(' '.join(sent))
```

- Lista nxt[a] conține repetițiile → alegerea uniformă respectă **frecvențele**
- Starea = cuvântul curent
- Corpusul nostru: 82 de cuvinte, 51 distincte
- Punctul „.” marchează începutul și sfârșitul propoziției

# Ce generează lanțul (corpusul cursului)

- „fiecare rulare a simulării dă un lanț markov are stări și tranziții”
- „fiecare rulare a unui lanț markov are stări și tranziții”
- „simularea rulează modelul de starea următoare depinde doar de multe ori”
- După „**modelul**” au urmat în corpus: „simplifică”, „de”, „.” → câte **1/3**

**Observația**

Fiecare pereche de cuvinte e plauzibilă, dar propoziția nu mai are sens: modelul **nu vede** decât un cuvânt înapoi.

# n-grame vs modele mari de limbaj (LLM)

## Model cu n-grame

- Tabel de **numărare**; context de 1–4 cuvinte
- Nu generalizează: o combinație nevăzută are probabilitatea 0
- Explozie: 50 000 de cuvinte → $2{,}5 \cdot 10^9$ bigrame posibile
- Util și azi: tastaturi, corectoare, baze de comparație

vs

## LLM (de ex. GPT, Claude)

- **Rețea neuronală** care calculează P(următorul token | context)
- Context de mii – milioane de tokeni
- Generalizează prin reprezentări învățate
- Formal tot „un token pe rând”, dar starea = **tot contextul**: nu mai e un tabel

# Modele de limbaj: o scurtă istorie

**1913** — **Markov**: vocale și consoane, lanț cu 2 stări, numărat de mână

**1948** — **Shannon**: texte generate cu n-grame de litere și de cuvinte

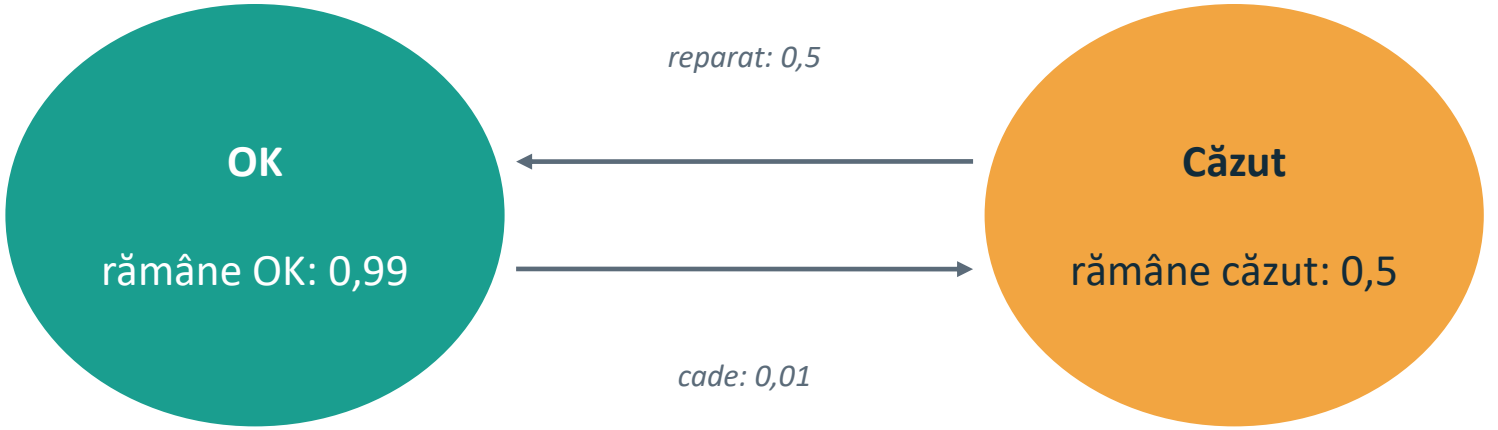
**1980–2010** — **n-grame** în recunoașterea vorbirii, traducere, tastaturi predictive

**2003** — **Bengio** et al.: primul model de limbaj cu rețea neurală

**2017** — **Transformer**: atenție asupra întregului context

**2020 →** — **LLM**: miliarde de parametri, context de mii – milioane de tokeni

# Fiabilitate: un server OK / căzut




Pas = o zi
a = 0,01 (cade)
b = 0,5 (e reparat)
MTBF = 100 zile
MTTR = 2 zile

*MTBF = timpul mediu între defecte = 1/a; MTTR = timpul mediu de reparație = 1/b*

# Disponibilitatea = π(OK)

$$A = b / (a + b) = MTBF / (MTBF + MTTR)$$

***exemplu*** A = 0,5 / 0,51 = 100 / 102 ≈ **98,04%** (simulare pe 100 de ani: 97,78%)

***pe an*** ≈ 172 de ore de indisponibilitate (≈ 7 zile)

***2 servere*** independente, cel puțin unul OK: $1 - (1 - A)^2 \approx$ **99,96%** (≈ 3,4 ore pe an)

Aceeași formulă cu 2 stări ca la vreme. Redundanța transformă „un noua” (98%) în „trei nouari și ceva” (99,96%).

# Disponibilitate și „nouari”

| Configurație | Disponibilitate | Oprire pe an | Clasa |
|---|---|---|---|
| **1 server (MTBF 100 z, MTTR 2 z)** | 98,04% | ≈ 172 ore | „un noua” |
| **2 servere independente** | 99,96% | ≈ 3,4 ore | „trei nouari” |
| **Țintă tipică cloud (SLA)** | 99,9% | ≈ 8,8 ore | „trei nouari” |
| **Țintă foarte strictă** | 99,999% | ≈ 5 minute | „cinci nouari” |

*Oprire pe an = (1 − A) × 8 760 de ore; redundanța ajută doar dacă defectele sunt independente*

MCMC

# Lanțuri Markov care eșantionează

1. Problema: eșantioane dintr-o distribuție cunoscută „până la o constantă”
2. Algoritmul Metropolis (1953)
3. Pasul propunerii, acceptarea, perioada de încălzire
4. Unde se folosește: statistică bayesiană, fizică, criptografie

# Problema: cum eșantionăm o distribuție complicată?

- Știm f(x) doar **până la o constantă** (tipic în statistica bayesiană)
- Transformarea inversă cere $F^{-1}$; acceptare-respingere cere o anvelopă bună (S2)
- În multe dimensiuni ambele devin impracticabile
- **Ideea MCMC**: construim un lanț Markov a cărui distribuție staționară este **f**
- Rulăm lanțul mult timp: stările vizitate sunt (aproximativ) eșantioane din f

↺ **Întoarcerea problemei**

Ora 2: dat lanțul, găsim π.
Acum: dată π, **construim** lanțul.

# Algoritmul Metropolis (1953)

Metropolis, Rosenbluth, Rosenbluth, Teller, Teller – calcul pe calculatorul MANIAC, Los Alamos:

**1**

**Start**

o valoare x oarecare

→

**2**

**Propunere**

y = x + zgomot (de ex. normal cu pasul σ)

→

**3**

**Raport**

r = f(y) / f(x): constanta necunoscută **se simplifică**

→

**4**

**Decizie**

acceptă y cu probabilitatea min(1, r); altfel rămâi în x

→

**5**

**Repetă**

păstrează stările; aruncă primele (**încălzire**)

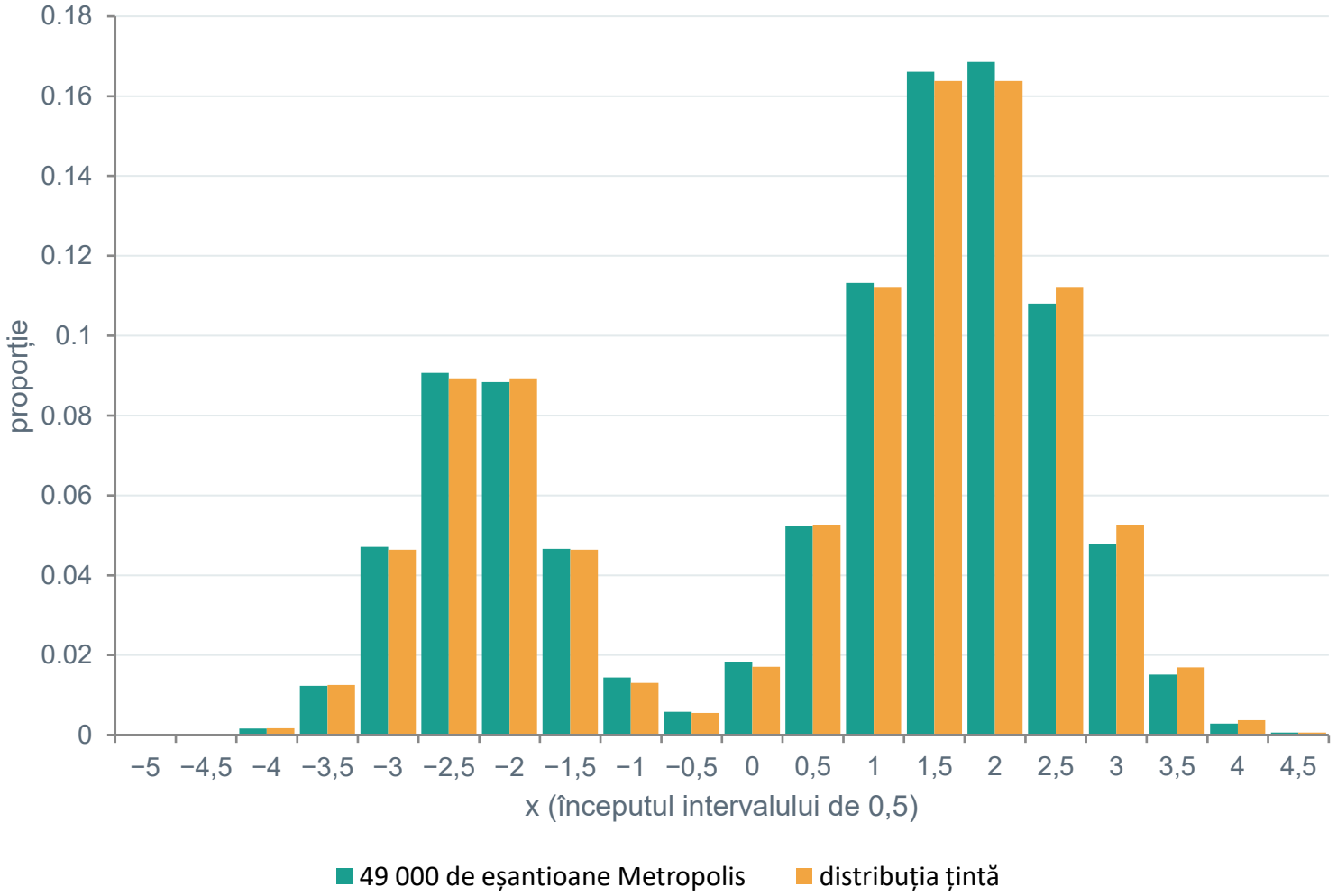
# Metropolis în 10 linii

```
import numpy as np
def f(x):   # 2 normale, fără constanta de normare
    return (0.3 * np.exp(-(x + 2)**2 / (2 * 0.6**2)) / 0.6
          + 0.7 * np.exp(-(x - 2)**2 / (2 * 0.8**2)) / 0.8)

rng = np.random.default_rng(16)
x, xs, acc = 0.0, [], 0
for i in range(50_000):
    y = x + rng.normal(0, 1.0)          # propunere
    if rng.random() < f(y) / f(x):      # acceptare
        x = y; acc += 1
    xs.append(x)
xs = np.array(xs[1000:])                # fără încălzire
```

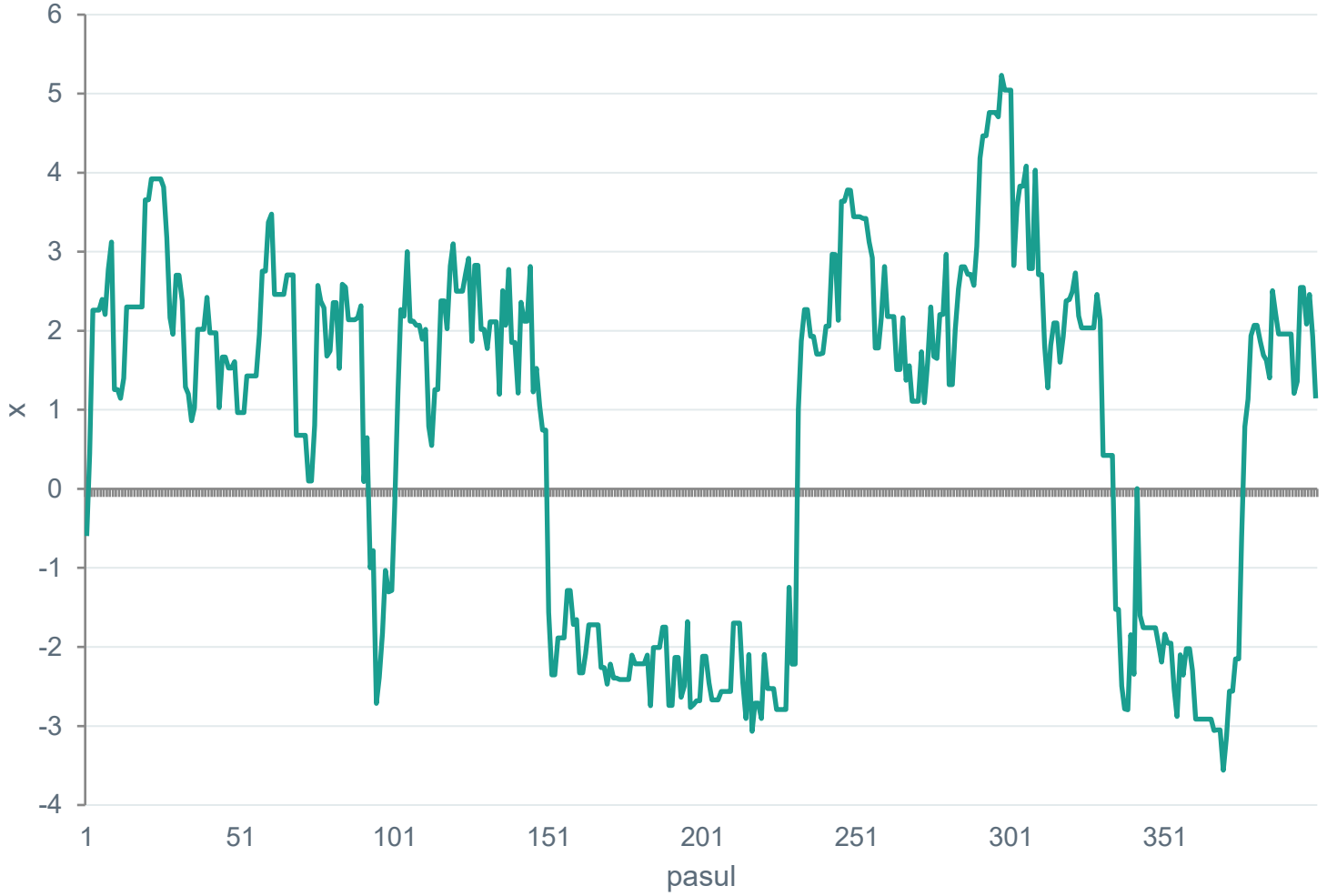
- Ținta: 30% în jurul lui −2, 70% în jurul lui +2
- **Nu** avem nevoie de constanta de normare
- Rata de acceptare: **63,7%**
- Media eșantioanelor: **0,777** (teoretic 0,8)

# Eșantioanele Metropolis vs distribuția țintă




- Cele două „cocoașe” sunt reproduse corect
- Fracțiunea x > 0: **0,693** (teoretic 0,7)
- Lanțul trece de la o cocoașă la alta prin zona puțin probabilă
- Eșantioanele sunt **corelate** – ca zilele de vreme

# Traiectoria lanțului (primii 400 de pași)




- Lanțul pornește din 0 și găsește repede zonele probabile
- Stă o vreme într-o cocoașă, apoi sare în cealaltă
- Primii pași depind de start → **perioada de încălzire** (burn-in)
- Graficul „trace” este primul diagnostic MCMC

# Mărimea pasului contează

**96%**

acceptare cu pasul **σ = 0,1**: aproape totul acceptat, dar lanțul abia se mișcă

**64%**

acceptare cu pasul **σ = 1**: un compromis bun pentru această țintă

**17%**

acceptare cu pasul **σ = 10**: propuneri prea îndrăznețe, respinse des

- Pas mic: acceptare mare, dar explorare lentă; pas mare: multe respingeri
- Reguli practice: acceptare de ordinul 20–50% în multe dimensiuni

# Unde se folosește MCMC

### 1 Statistică bayesiană

Stan, PyMC: distribuția parametrilor unui model, dată fiind o mulțime de date.

### 2 Fizică

Modelul Ising (magnetism), simulări moleculare – aplicația originală din 1953.

### 3 Criptanaliză

Diaconis (2009): un mesaj cifrat prin substituție, decodat cu MCMC și statistici de bigrame.

### 4 Învățare automată

Modele generative, estimarea incertitudinii, eșantionare în spații uriașe.

# Teaser: procese de decizie Markov (MDP)

## Proces de decizie Markov

Un lanț Markov în care, la fiecare pas, un **agent alege o acțiune** a: tranziția devine P(s’ | s, a) și primește o **recompensă** r(s, a). Scopul: o **politică** a = π(s) care maximizează recompensa totală.

- Exemplu: autoscaling – stare = încărcarea, acțiune = adaug / scot un server, recompensă = −cost − penalizări de latență
- Bellman (1957): programarea dinamică; baza **învățării prin întărire** (RL - Reinforcement Learning)
- RL (Reinforcement Learning) + simulatoare: roboți, jocuri, conducere autonomă (săptămâna 10)

# Greșeli frecvente

- Stări care **nu** au proprietatea Markov (lipsește memoria necesară din stare)
- Rânduri care nu au suma 1; confuzia rând = „de unde” / coloană = „unde”
- Folosirea lui π pentru un lanț **periodic** sau **reductibil** ca și cum ar fi limita
- Timpul de ședere calculat cu $\pi_i$ în loc de $P_{ii}$
- MCMC: fără perioadă de încălzire; eșantioane tratate ca independente
- Matrice estimate din **prea puține** tranziții (starea „Căzut”: 31 de observații)

✓ **Verificare rapidă**

Simulați un drum lung și comparați fracțiunile de timp cu π calculat. Dacă diferă, ceva e greșit.

# Verificare

? În PageRank cu factorul de amortizare **d = 0,85**, cu ce probabilitate sare internautul, la fiecare pas, la o pagină **aleasă uniform**?

A 0,85

B 0,15

C 0,5

D 1/n

# Verificare

? Un serviciu are **MTBF = 99 de zile** și **MTTR = 1 zi**. Care este disponibilitatea lui pe termen lung?

A 1%

B 90%

C 99%

D 99,99%

# Verificare

?

Într-un corpus, cuvântul „date” apare de **50** de ori și este urmat de „reale” de **20** de ori. Cât este P̂(„reale” | „date”) în modelul cu bigrame?

A 0,2

B 0,4

C 0,5

D 20

# Verificare

?

În algoritmul Metropolis, starea curentă are f(x) = **0,20**, iar propunerea are f(y) = **0,05**. Cu ce probabilitate **acceptăm** propunerea?

A 0

B 0,05

C 0,25

D 1

# Rezumatul săptămânii 3

- ✓ Plimbarea aleatoare: distanța ~ $\sqrt{n}$; în 2D $\langle r^2 \rangle = 4Dt$ → difuzia (micro → macro)
- ✓ Ruina jucătorului: P = k/N (joc corect); tendința negativă câștigă pe termen lung
- ✓ Lanț Markov: stări + **matrice de tranziție** (rânduri cu suma 1); $\pi^{(n)} = \pi^{(0)}P^n$
- ✓ **Distribuția staționară** $\pi = \pi P$: fracțiunea de timp; 2 stări: $\pi_1 = b/(a + b)$
- ✓ Ergodic → π unică; periodic → oscilează; absorbant → $N = (I - Q)^{-1}$
- ✓ PageRank = π a internautului aleator (d = 0,85); n-grame ≠ LLM
- ✓ Disponibilitate = MTBF/(MTBF + MTTR); **Metropolis**: acceptă cu min(1, f(y)/f(x))

Săptămâna viitoare

# Teoria cozilor și performanța sistemelor

- Sosiri Poisson, notația Kendall, legea lui Little
- M/M/1 ca lanț Markov: de ce latența explodează la încărcare mare
- Servere web, cloud, servirea modelelor LLM

*De ce un server ocupat 90% din timp răspunde de 5 ori mai încet decât unul ocupat 50%?*

# Bibliografie pentru săptămâna 3


- Ș. Holban – Tehnici de modelare și simulare, cap. 5 (Modele Markov) și prezentarea „Markov Chains”
- Cernazean – Modelling and Simulation, Lecture 3 (plimbarea aleatoare, difuzia)
- O. Häggström – Finite Markov Chains and Algorithmic Applications, Cambridge University Press, 2002
- S. M. Ross – Introduction to Probability Models, Academic Press – capitolul despre lanțuri Markov
- S. Brin, L. Page – The Anatomy of a Large-Scale Hypertextual Web Search Engine, 1998
- B. Hayes – First Links in the Markov Chain, American Scientist, 2013
- N. Metropolis et al. – Equation of State Calculations by Fast Computing Machines, J. Chem. Phys., 1953
- P. Diaconis – The Markov Chain Monte Carlo Revolution, Bulletin of the AMS, 2009
- Prezentare realizata cu ajutorul Claude Fable 5.1 și Claude Opus 5.5